

llm-from-scratch 教學文件

從零手刻 Transformer

每一條公式與反向傳播都自己寫

github.com/GarfieldHuang/llm-from-scratch

2026-09-06

目錄

1. 符號說明
2. 全景圖
3. tokenizer
4. embedding
5. 反向傳播
6. adamw
7. 常見誤解
8. 權重與tokenizer綁定
9. rmsnorm
10. attention
11. ffn
12. 完整執行紀錄

符號說明

其他文件裡用到的所有符號都在這裡。**建議先看這份**，尤其是 $\odot$ 和 $@$ 那一節——這兩個都叫「乘」，但完全是兩回事，混淆的話後面每條公式都會讀錯。

可執行的對照示範：

```
python tools/demo_operators.py
```

最容易混淆的兩個： $\odot$ 與 $@$

$\odot$ 逐元素相乘 (Hadamard product)

$$(A \odot B)_{ij} = A_{ij} \times B_{ij}$$

位置對位置相乘，形狀不變。

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$
$$A \odot B = \begin{bmatrix} 1 \times 5 & 2 \times 6 \\ 3 \times 7 & 4 \times 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

左上角那格 $= 1 \times 5 = 5$ 。每一格只跟「同一格」的對手相乘，跟其他格完全無關。

$@$ 矩陣乘法 (matrix multiplication)

$$(A @ B)_{ij} = \sum_k A_{ik} \times B_{kj}$$

左邊的「列」跟右邊的「行」做點積。

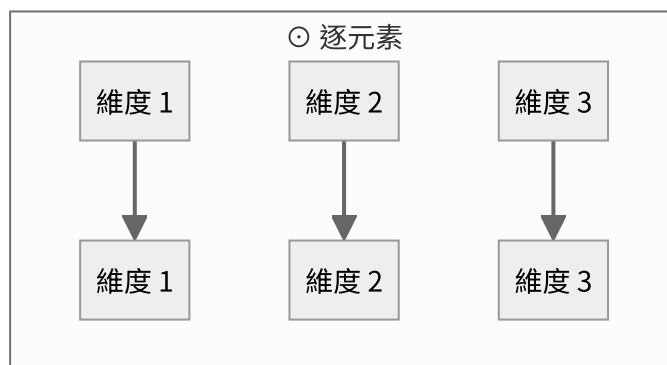
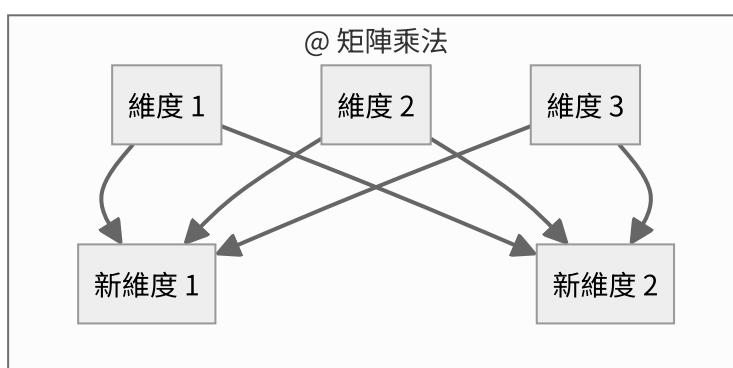
$$A @ B = \begin{bmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

左上角那格 $= 1 \times 5 + 2 \times 7 = 19$ 。用到了一整列和一整行，也就是所有維度。

同樣的輸入，結果完全不同

	$A \odot B$	$A @ B$
左上角	5	19
算法	一對一相乘	一整列 · 一整行
形狀	必須相同，輸出也相同	$(m, k) @ (k, n) \rightarrow (m, n)$
會不會混合維度	不會	會

為什麼「會不會混合維度」是關鍵



⊙ 的每一條線都是平行的，資訊不會橫向流動。@ 的每個輸出都吃進了所有輸入。

這決定了它們在模型裡的角色：

符號	出現在哪	做什麼
@	Linear : $y = x @ W^T$	把 d 維換成另外 d_{out} 維
@	Attention : QK^T	每個位置跟每個位置做點積
@	輸出層 : hE^T	隱藏狀態跟每個詞向量做點積
⊙	RMSNorm : $y = g \odot \hat{x}$	每個維度乘上自己的縮放係數
⊙	SwiGLU : $m = s \odot u$	閘門：每個維度各自決定放行多少
⊙	RoPE : $x \odot \cos$	每個維度乘上自己的旋轉係數

⊙ 永遠不會把不同維度的資訊混在一起。所以模型的「學習能力」幾乎全部來自 @，⊙ 只負責調整強度。

這也解釋了為什麼「疊很多個 ⊙」沒有用——不管乘幾次，維度 1 永遠只影響維度 1。

形狀規則

x shape (4, 8)

$x @ W.T \rightarrow (4, 3)$ $(4, 8) @ (8, 3)$ 最後一維被換掉了

$x \odot g \rightarrow (4, 8)$ $(4, 8) \odot (8,)$ 形狀完全不變

$x \odot g$ 用到了廣播 (broadcasting)：g 只有 8 個數字，會自動複製 4 份，對 x 的每一列各做一次逐元素相乘。

反向傳播的規則也不一樣

$$y = a \odot b \implies \frac{\partial L}{\partial a} = \frac{\partial L}{\partial y} \odot b, \quad \frac{\partial L}{\partial b} = \frac{\partial L}{\partial y} \odot a$$
$$y = x @ W^T \implies \frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} @ W, \quad \frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial y} \right)^T @ x$$

兩個都符合同一句話：對其中一邊微分，得到的就是另一邊。

`tools/demo_operators.py` 用有限差分驗證了這兩條，誤差 10^{-10} 等級。

其他符號

運算

符號	名稱	意思
$\cdot$	點積	兩個向量對應相乘後加總，得到一個 純量 ： $a \cdot b = \sum_k a_k b_k$
$\otimes$	外積	兩個向量產生一個 矩陣 ： $(a \otimes b)_{ij} = a_i b_j$
$A^\top$	轉置	列與行互換： $(A^\top)_{ij} = A_{ji}$
$\sum_k$	求和	對指標 k 把所有項加起來
$\ x\ $	範數	向量長度： $\sqrt{\sum_k x_k^2}$
$\leftarrow$	賦值	「用右邊的值取代左邊」，不是等式

$\cdot$ 和 $@$ 的關係： $@$ 就是把 $\cdot$ 做很多次。 $(A @ B)_{ij}$ 就是「 A 的第 i 列」 $\cdot$ 「 B 的第 j 行」。

微分

符號	意思
$\frac{\partial L}{\partial x}$	L 對 x 的偏導數。白話：「把 x 動一丁點， L 會變多少」
$\frac{\partial L}{\partial \mathbf{x}}$	對整個向量微分，結果跟 $\mathbf{x}$ 同形狀
$f'(z)$	單變數函數的導數
δ_{ij}	Kronecker delta： $i = j$ 時為 1，否則為 0

程式碼裡為了簡潔， $\frac{\partial L}{\partial x}$ 一律寫成 `dx`。

函數

符號	定義
$\sigma(z)$	sigmoid： $\frac{1}{1 + e^{-z}}$ ，把任意實數壓到 (0, 1)
$\text{SiLU}(z)$	$z \sigma(z)$ ，也叫 Swish
$\text{softmax}(\mathbf{z})_i$	$\frac{e^{z_i}}{\sum_k e^{z_k}}$ ，把一排分數變成機率（和為 1）
$\text{mean}(x)$	平均值

形狀與維度

符號	意思	本專案的典型值
B	batch size，一次處理幾段文字	64
T	序列長度，一段文字幾個 token	128
d	模型維度 (hidden size)	256
h	attention head 數	4
d_{head}	每個 head 的維度， $= d/h$	64
d_{ff}	FFN 的中間維度	672
$ \mathcal{V} $	詞表大小	6400
N	一個 batch 裡有效的預測位置數	$B \times T$

張量形狀寫成 (B, T, d) ，最後一維永遠是「特徵維度」。

常見的讀法陷阱

① $W^{\top}$ 出現在哪一邊

程式碼寫 `x @ W.T`，因為 PyTorch 的慣例是 W 的形狀為 $(d_{\text{out}}, d_{\text{in}})$ 。所以要先轉置才對得上。數學文獻常寫成 Wx (W 在左)，兩者是同一件事，只是排列不同。

② $\frac{\partial L}{\partial W}$ 的形狀

永遠跟 W 一樣。梯度是「每個參數各自的斜率」，所以有幾個參數就有幾個梯度。

③ $\leftarrow$ 不是 $=$

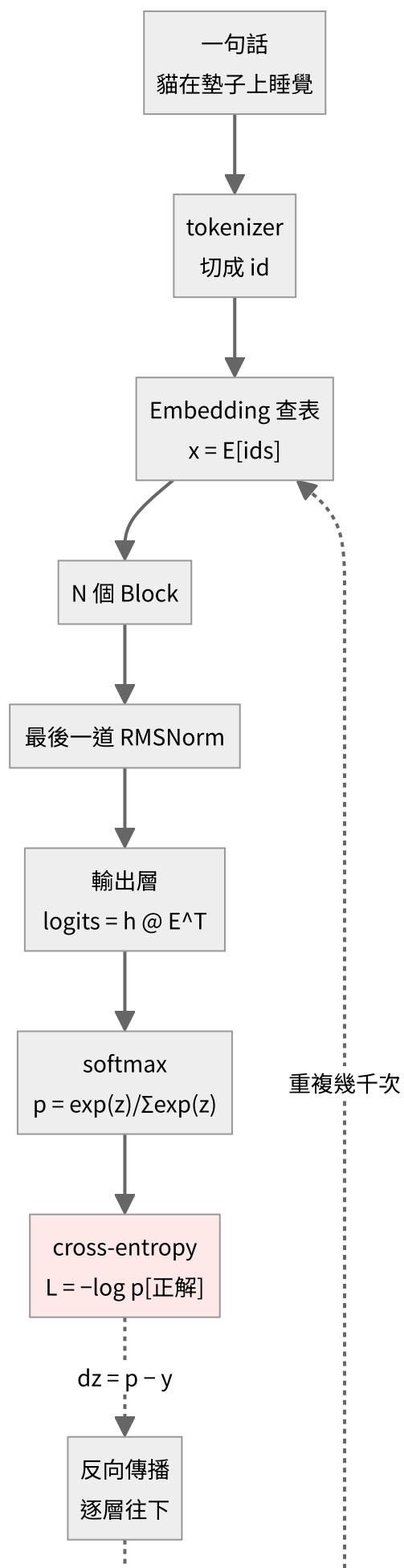
$$\theta \leftarrow \theta - \eta g$$

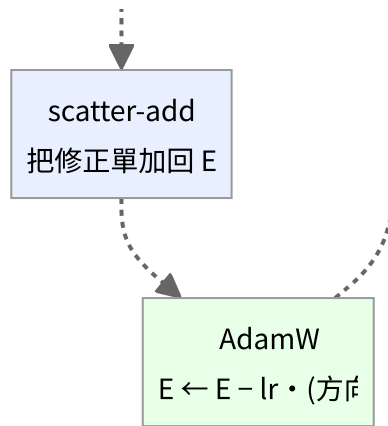
這是「更新」：用右邊算出來的值取代左邊。寫成 $\theta = \theta - \eta g$ 在數學上是個無解的方程式。

全景圖

一句話進去，到權重被改動，中間到底發生了什麼。

完整流程

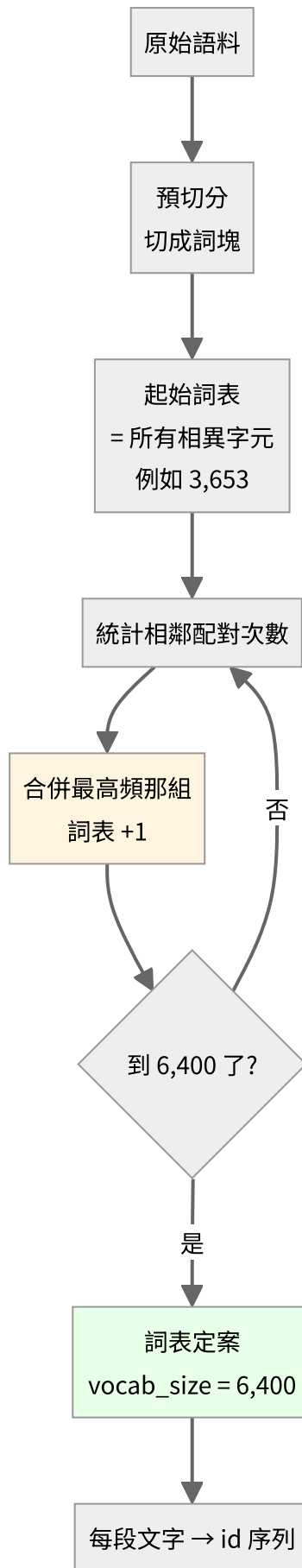




實線是前向，虛線是反向。注意那條回到最上面的迴圈——**訓練就是這個迴圈跑幾千次。**

第 0 步：文字怎麼變成 id

上圖第二格的「tokenizer」其實是一個**獨立於模型之外**的階段。它在訓練開始前就先跑完，之後整個訓練過程都不會再變動。



這裡沒有任何學習，純粹是貪婪的頻率統計。沒有梯度、沒有神經網路。

$$|\mathcal{V}| = |\mathcal{V}_0| + M$$

實測：

起始詞表 3,653 (4 特殊 + 3,649 字元)

目標 6,400 → 需要合併 2,747 次

$$3,653 + 2,747 = 6,400$$

它為什麼是全景圖的一部分——**tokenizer 決定了模型的架構參數**：

決定了什麼	為什麼
詞表大小 $ \mathcal{V} $	直接等於 embedding 的列數
embedding 參數量	$ \mathcal{V} \cdot d$ ，這裡是 $6400 \times 256 = 1.6 \text{ M}$
logits 張量大小	$B \times T \times \mathcal{V} $ ，訓練時最大的中間張量
序列長度	BPE 讓序列比字元級短約 30%，而 attention 是 $O(n^2)$ ，計算量差更多

切分效果的直接影響（同一個模型、同一份語料）：

原文 機器學習是人工智慧的一個分支

字元級 ['機', '器', '學', '習', '是', '人', '工', '智', '慧', '的', '一', '個', '分', '支'] 14 tokens

BPE ['機器學習', '是', '人', '工智慧', '的一個', '分', '支'] 7 tokens

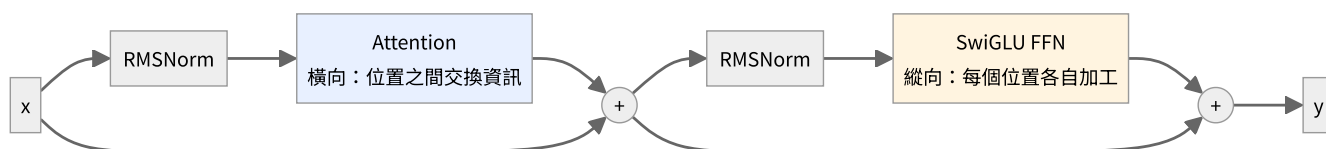
機器學習 變成一個 token。這代表模型不必再從四個字元學會它們常常一起出現，那件事在 tokenizer 階段就處理掉了。

細節見 02-tokenizer，程式碼在 [tokenizer/bpe.py](#)。

注意：換 tokenizer 之後，loss 的數值**不能直接互相比較**。字元級 3,195 詞表跑出 loss 2.24，BPE 6,400 詞表跑出 2.76，但後者每個 token 攜帶的資訊量更大，猜對更難。要公平比較得換算成 bits-per-character：

$$\text{bpc} = \frac{L \cdot N_{\text{token}}}{N_{\text{char}} \cdot \ln 2}$$

一個 Block 的內部



寫成公式：

$$\mathbf{h} = \mathbf{x} + \text{Attention}(\text{RMSNorm}(\mathbf{x}))$$

$$\mathbf{y} = \mathbf{h} + \text{FFN}(\text{RMSNorm}(\mathbf{h}))$$

三件事要記住：

1. 順序是 **Norm** → **Attention** → **Norm** → **FFN**。不是 FFN 先。
2. 那兩個 + 是殘差。前向 $\mathbf{y} = \mathbf{x} + f(\mathbf{x})$ ，反向

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} + f' \left(\frac{\partial L}{\partial \mathbf{y}} \right)$$

那個孤零零的 $\partial L / \partial \mathbf{y}$ 是「直達路徑」，梯度可以完全不衰減地穿過去。沒有殘差的話，20 層以上基本訓不動。

3. **Attention 橫向**，**FFN 縱向**。Attention 是唯一讓不同位置互通的地方；FFN 對每個位置各做各的。

資料的形狀怎麼變

以 $B = 8$ 、 $T = 64$ 、 $d = 256$ 、 $|\mathcal{V}| = 6400$ 為例：

階段	形狀	說明
token ids	(8, 64)	整數
查表後	(8, 64, 256)	每個位置一個 256 維向量
每個 Block 之後	(8, 64, 256)	形狀不變 ，只是內容被加工
logits	(8, 64, 6400)	每個位置對整個詞表的分數
loss	()	一個純量

注意 logits 那一步：

$$8 \times 64 \times 6400 = 3.27 \times 10^6$$

這通常是整個模型**最大的中間張量**，訓練時的記憶體大戶。

參數住在哪裡

跑 `python train.py` 會印出來，例如：

模型結構

=====	
embedding	204,480
block0	47,232
block1	47,232
final_norm	64
lm_head (與 embedding 共用)	0

總計	299,008
embedding 佔比	68.4%

小模型的 embedding 佔比會很誇張，因為兩者的量級是

$$\underbrace{|\mathcal{V}| \cdot d}_{\text{embedding}} \quad \text{vs} \quad \underbrace{O(d^2)}_{\text{每一層}}$$

而小模型的 $|\mathcal{V}| \gg d$ 。真實的大模型 d 大得多，embedding 佔比會降到 5% 以下。

前向與反向的對稱

每一層的 `backward` 都在回答同一個問題：「我的輸出該怎麼動」→「我的輸入該怎麼動」。

層	前向	反向
Linear	$\mathbf{y} = \mathbf{x}W^\top$	$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}}W, \frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial \mathbf{y}}\right)^\top \mathbf{x}$
Embedding	$\mathbf{x} = E_v$ (gather)	$\frac{\partial L}{\partial E_v} = \sum_{t: \text{id}_t=v} G_t$ (scatter-add)
RMSNorm	$\hat{x} = x/r$	$\frac{1}{r} (a - \hat{x} \text{mean}(a \odot \hat{x}))$
RoPE	旋轉 $+\theta$	旋轉 $-\theta$
softmax + CE	$\mathbf{p} = \text{softmax}(\mathbf{z})$	$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}$
殘差 $\mathbf{y} = \mathbf{x} + f(\mathbf{x})$	分岔	相加

最後一列是通則：一個張量被用了 k 次，它的梯度就是那 k 條路的總和。

$$\frac{\partial L}{\partial \mathbf{x}} = \sum_{i=1}^k \frac{\partial L}{\partial \mathbf{x}} \Big|_{\text{第 } i \text{ 條路}}$$

下一步

- 想知道詞表大小怎麼定 → 02-tokenizer
- 想知道 embedding 怎麼被訓練 → 03-embedding
- 想跟著梯度走一遍 → 04-反向傳播

詞表大小是怎麼決定的

「minimind 的詞表 6400 個 token，這個數字是哪來的？」

不是算出來的，是你自己挑的停止條件。

$$|\mathcal{V}| = \underbrace{|\mathcal{V}_0|}_{\text{起始符號數}} + \underbrace{M}_{\text{合併次數}}$$

BPE 每合併一次，詞表就多一個。想要 $|\mathcal{V}| = 6400$ ，就合併到總數變成 6400 為止。

實際跑一次的數字

```
python tests/test_bpe.py
```

詞塊 183,640 個（相異 74,440 個），字元 546,000 個

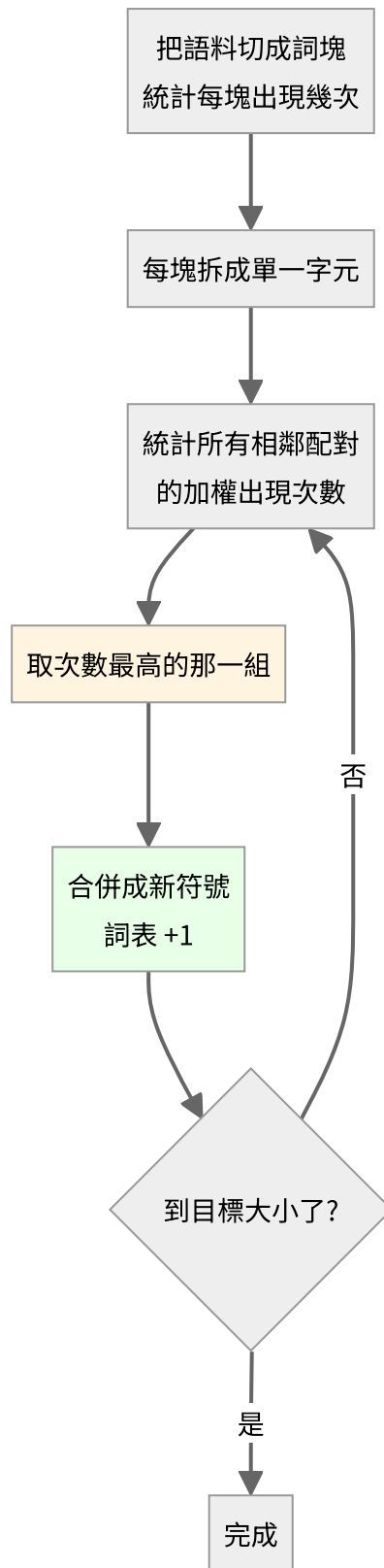
起始詞表 3,653（4 特殊 + 3,649 字元）

目標 6,400 → 需要合併 2,747 次

$$3,653 + 2,747 = 6,400$$

看最後一行。就這麼直接。

BPE 在做什麼



沒有任何學習、沒有梯度、沒有神經網路。純粹是貪婪的頻率統計。

合併過程長什麼樣

`traces/bpe_training.txt` 會記下每一次合併：

```
合併 #1      ('可', '以') 出現 2,362 次 -> 新 token '可以'    詞表 3,654
合併 #2      ('我', '們') 出現 1,988 次 -> 新 token '我們'    詞表 3,655
...
合併 #1000   ('歌', '詞') 出現   40 次 -> 新 token '歌詞'    詞表 4,653
合併 #2000   ('排', '放') 出現   21 次 -> 新 token '排放'    詞表 5,653
```

注意**出現次數一路遞減**：2362 → 1988 → ... → 40 → 21。

這給了一個很實用的判斷：**如果最後幾次合併的次數已經降到個位數，代表詞表開太大了**，那些 token 學不到什麼東西。反過來，如果停下來的時候 次數還有好幾百，代表還可以再擴。

效果

500 段，共 140,502 字元

字元級 140,502 tokens 1.000 token/字

BPE 99,202 tokens 0.706 token/字

BPE 讓序列短了 29.4%

切分對照：

原文 機器學習是人工智慧的一個分支

字元級 ['機','器','學','習','是','人','工','智','慧','的','一','個','分','支']

BPE ['機器學習','是','人','工智慧','的一個','分','支']

機器學習 變成一個 token 了。序列變短的直接好處：

- 同樣的 context window 能裝更多內容
- Attention 是 $O(n^2)$ ，序列短一半，注意力的計算量剩四分之一

為什麼順序不能亂

編碼的時候要**照訓練時學到的順序**套用合併：

'機器學習' 這個 token 存在的前提，是 '機器' 和 '學習' 已經先被合併出來了。

所以 `bpe.py` 裡每個合併都記了 `rank`（第幾次學到的），編碼時永遠先套 rank 最小的。

這份實作與生產級的差別

	這裡	GPT-2 / Qwen / minimind
起始符號	語料裡的 字元	256 個 byte
OOV	用 <code><unk></code>	不可能有（任何字都能拆成 byte）
追蹤檔可讀性	每個符號都是你認得的字	生僻字會變成看不懂的 byte 碎片

byte-level 的代價，在前面的實驗裡量過：用簡體訓的 tokenizer 去吃繁體語料時，「這」會被拆成兩個 byte 碎片，序列長度膨脹 1.67 倍。因為那些繁體字**根本不在詞表裡**。

這也是為什麼換語言、換領域時，tokenizer 常常得重訓。

一個容易忽略的連結

詞表大小直接決定了 embedding 的參數量：

$$|\theta_{\text{embed}}| = |\mathcal{V}| \times d$$

$6400 \times 768 = 4,915,200$ 。詞表開兩倍，embedding 就大兩倍。

而且因為 weight tying，同一個矩陣還兼任輸出層——所以詞表大小也直接決定了最後那個 logits 張量有多大：

$$\text{logits} \in \mathbb{R}^{B \times T \times |\mathcal{V}|}$$

那通常是訓練時**最大的中間張量**。以 $B = 64$ 、 $T = 128$ 、 $|\mathcal{V}| = 6400$ 為例，光這一個張量就有 5.24×10^7 個數字。

詞表大小不只是 tokenizer 的參數，它同時是模型的架構參數。

Embedding 與 scatter-add

對應程式碼：`scratch/embedding.py`

它不是資料，是權重

最容易誤會的一點： E 看起來像一張查詢表，所以直覺會把它歸類成「輸入」。

但它在 `model.params()` 裡跟 `q_proj.W` 排在一起，用同一個 optimizer、同樣的 η 、同樣的更新規則。跑 `train.py` 印出的參數表就是證據：

```
model.embed.E          204,480    ← 就是它
block0.attn.q_proj.W    16,384
block0.ffn.gate_proj.W  43,776
...
optimizer 拿到 20 個參數張量
```

「embedding 有一套自己的訓練方法」——這件事不存在。

查表 = one-hot 矩陣乘法

$$\mathbf{x} = \text{onehot}(v) E, \quad E \in \mathbb{R}^{V \times d}$$

$\text{onehot}(v)$ 只有第 v 個元素是 1、其餘全 0。乘出來就是把 E 的第 v 列抄出來：

$$x_j = \sum_{k=1}^V \text{onehot}(v)_k E_{kj} = E_{vj}$$

既然 $\frac{V-1}{V} = 99.98\%$ 的乘法都是「乘以 0」，實作當然直接索引：

```
x = E[ids]
```

兩者結果逐位元相同，實測查表快 20 倍以上、省 8 倍記憶體。

把它看成矩陣乘法很重要——因為反向就是從這裡推出來的。

反向：scatter-add

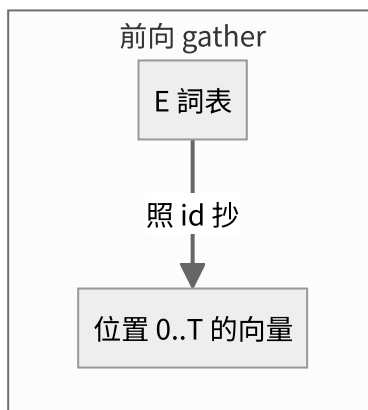
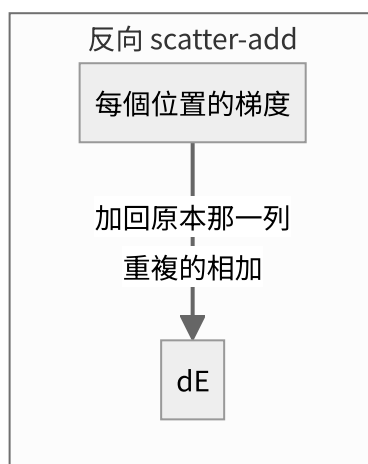
把整批寫成矩陣形式 $X = O E$ (O 是常數 one-hot 矩陣)，套用線性層的標準結果：

$$\frac{\partial L}{\partial E} = O^\top G, \quad G = \frac{\partial L}{\partial X}$$

$O^\top$ 的第 v 列，在「第 t 個位置的 id 等於 v 」的地方是 1，其餘是 0。展開得到：

$$\frac{\partial L}{\partial E_v} = \sum_{t: \text{id}_t=v} G_t$$

白話：把每個位置的梯度，加回它當初查表的那一列；同一列有多筆就相加。



由此直接得到兩個常被誤會成「特殊規則」的性質——其實都只是算出來的結果：

現象	為什麼
同一個 token 出現 k 次 → 那一列收到 k 筆梯度 相加	$O^\top$ 那一列有 k 個 1，矩陣乘法自然就加起來了
沒出現過的 token → 梯度是 0	$O^\top$ 那一列全是 0

追蹤檔裡的實際數字

embed . backward (scatter-add, 輸入側)

公式: $dE[v] += \sum_{t: id_t = v} dx[t]$

「id 4」在這一批出現 27 次

看第 0 維，這 27 筆修正單相加：

位置 1 $dx[0] = +1.09540625e-02$

位置 12 $dx[0] = +5.40754059e-03$

...

這 27 筆相加 = $+1.67316496e-02$

但梯度其實有兩條路

這是最容易混淆的地方。因為 weight tying (E 同時是輸出層的權重)，它會收到**兩份**梯度。

輸出側那條有封閉解。因為 $\text{logits}_t = \mathbf{h}_t E^\top$ ：

$$\left. \frac{\partial L}{\partial E_v} \right|_{\text{out}} = \frac{1}{N} \sum_t (p_t[v] - y_t[v]) \mathbf{h}_t$$

實測：

來源	$\partial L / \partial E_4[0]$	有幾列非零
輸入側 (scatter-add)	+0.01673	272/3195 ← 稀疏
輸出側 (weight tying)	+0.00500	3195/3195 ← 稠密
合計	+0.02174	3195/3195

	來自哪裡	稀疏還是稠密
輸入側	查表這條路，backprop 傳回來的	稀疏，只有出現過的 token
輸出側	$\text{logits} = \mathbf{h} E^\top$ ，softmax 那條路	稠密，每一步每一列都被碰到

所以「embedding 的梯度是稀疏的」這句話，**在有 weight tying 的模型上並不成立**。

想親眼驗證：把 `TinyLM(..., tie_weights=False)` 跑一次，非零列就會剩下 272。

注意這裡的點積是「隱藏狀態 · 詞向量」，**不是**「詞向量 · 詞向量」。訓練過程中從來沒有計算過任何兩個詞向量之間的相似度——cosine 只是我們事後檢查用的量尺。

「離散的 id 怎麼微分？」

不對 id 微分。

	是什麼	離散/連續	微分嗎
id = 4	資料	離散	不用，也不能
$E_{4,0} = -0.0153$	參數	連續實數	對它微分

梯度問的是：

$$\frac{\partial L}{\partial E_{4,0}} = \lim_{\varepsilon \rightarrow 0} \frac{L(E_{4,0} + \varepsilon) - L(E_{4,0} - \varepsilon)}{2\varepsilon}$$

這個問題跟 id 是不是離散**完全無關**。id 只決定「哪些數字參與計算」。

沒參與的，斜率算出來自然是 0：

$$f(a, b, c) = a + b \implies \frac{\partial f}{\partial c} = 0$$

不是「不能微分」，是「算出來就是 0」。

有限差分實測 ($\varepsilon = 10^{-3}$ ，句子含重複的「在」)：

格子	$\frac{L_+ - L_-}{2\varepsilon}$	公式	說明
$E_{2,0}$ (在)	0.246048	0.246000	查兩次，斜率加倍
$E_{0,0}$ (貓)	0.123024	0.123000	查一次
$E_{1,0}$ (狗)	0.000000	0.000000	沒被查到

「狗」那一列 $L(w + \varepsilon)$ 和 $L(w - \varepsilon)$ **完全相等**—— 那個數字沒參與計算，動它 L 不會變。

`tests/test_layers.py` 對這一層的驗證結果：相對誤差 6.80×10^{-14} 。

延伸：離散在哪裡才真的會咬人

情境	可微嗎
訓練 (teacher forcing)	可微。id 是資料集給定的常數
生成 ($\arg \max$ / 取樣選下一個 token)	不可微，梯度在這裡斷掉

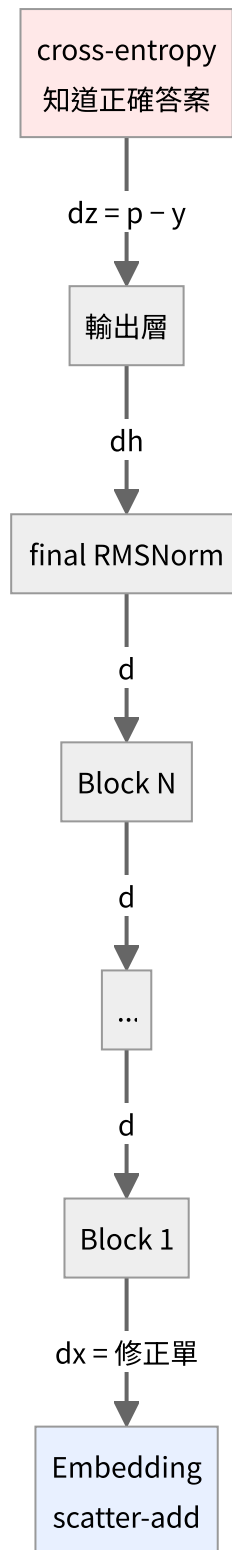
第二種才是真麻煩——這正是 RLHF / GRPO 那類「用生成結果的好壞來訓練」的方法 需要 policy gradient 之類特殊技巧的原因。

訓練 embedding 完全不碰這個問題。

反向傳播全鏈

從 $p - y$ 一路走回 E 。

整條鏈只有一個地方知道「對錯」



只有最上面那一格含有「答案」的資訊。往下每一層都只是在做微分，不知道任務是什麼、不知道「貓是動物」。

第一棒： $\frac{\partial L}{\partial z} = p - y$

softmax 和 cross-entropy 分開推都很醜（會出現 $V \times V$ 的 Jacobian），但合起來推會神奇地簡化。

softmax 的 Jacobian：

$$\frac{\partial p_i}{\partial z_j} = p_i (\delta_{ij} - p_j)$$

接上 cross-entropy $L = -\log p_y$ ：

$$\frac{\partial L}{\partial z_j} = \sum_i \frac{\partial L}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_j} = -\frac{1}{p_y} \cdot p_y (\delta_{yj} - p_j) = p_j - \delta_{yj}$$

整個 Jacobian 消失，只剩：

$$\frac{\partial L}{\partial \mathbf{z}} = \mathbf{p} - \mathbf{y}$$

其中 $\mathbf{y}$ 是正解的 one-hot。翻成白話：**正解那一格的分數要拉高，其他全部壓低。**

追蹤檔裡看到的：

token id	p	y	$\frac{p-y}{N}$	作用
213	0.35341	0	+0.003534	壓下去
57	0.11518	0	+0.001152	壓下去
4	0.11176	1	-0.008882	拉上來（正解）

（除以 N 是因為 loss 對所有有效位置取了平均。）

中間每一棒都在做同一件事

每一層的 `backward` 都回答：「我的輸出該怎麼動」→「那我的輸入該怎麼動」。翻譯的方法就是用自己的導數。

Linear： $\mathbf{y} = \mathbf{x}W^\top$

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} W \quad \frac{\partial L}{\partial W} = \left(\frac{\partial L}{\partial \mathbf{y}} \right)^\top \mathbf{x}$$

注意 $\partial L / \partial \mathbf{x}$ 用 W 、 $\partial L / \partial W$ 用 $\mathbf{x}$ —— **點積對其中一邊微分，得到的就是另一邊。** 這條規律整份程式碼裡反覆出現。

逐元素的非線性： $\mathbf{z} = f(\mathbf{a})$

$$\frac{\partial L}{\partial \mathbf{a}} = \frac{\partial L}{\partial \mathbf{z}} \odot f'(\mathbf{a})$$

SiLU 的導數（用 $\sigma' = \sigma(1 - \sigma)$ 推）：

$$\text{SiLU}(z) = z \sigma(z), \quad \text{SiLU}'(z) = \sigma(z)(1 + z(1 - \sigma(z)))$$

殘差： $\mathbf{y} = \mathbf{x} + f(\mathbf{x})$

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} + f' \left(\frac{\partial L}{\partial \mathbf{y}} \right)$$

那個孤零零的 $\partial L / \partial \mathbf{y}$ 就是「直達路徑」——梯度可以完全不衰減地穿過去。**沒有殘差的話，20 層以上基本訓不動。**

通則

前向分岔 $\implies$ 反向相加

$\mathbf{x}$ 在 SwiGLU 裡同時餵給 gate 和 up 兩路，所以

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{x}} \Big|_{\text{gate}} + \frac{\partial L}{\partial \mathbf{x}} \Big|_{\text{up}}$$

Attention 裡 $\mathbf{x}$ 餵給 Q、K、V 三路，所以三份相加。

RMSNorm：一個需要商法則的推導

前向：

$$r = \sqrt{\frac{1}{n} \sum_k x_k^2 + \varepsilon}, \quad \hat{x} = \frac{x}{r}, \quad y = g \odot \hat{x}$$

先求 r 對 x 的偏導：

$$\frac{\partial r}{\partial x_j} = \frac{1}{2} (\cdot)^{-1/2} \cdot \frac{2x_j}{n} = \frac{x_j}{n r}$$

y_i 透過兩條路依賴 x_j （分子的 x_i 、分母 r 裡的 x_j ）：

$$\frac{\partial y_i}{\partial x_j} = g_i \left[\frac{\delta_{ij}}{r} - \frac{x_i}{r^2} \cdot \frac{\partial r}{\partial x_j} \right] = g_i \left[\frac{\delta_{ij}}{r} - \frac{x_i x_j}{n r^3} \right]$$

令 $a_i = \frac{\partial L}{\partial y_i} g_i$ ，連鎖律加總後化簡：

$$\boxed{\frac{\partial L}{\partial x} = \frac{1}{r} \left(a - \hat{x} \cdot \text{mean}(a \odot \hat{x}) \right)}$$

$$\frac{\partial L}{\partial g} = \sum_{\text{所有位置}} \frac{\partial L}{\partial y} \odot \hat{x}$$

第二項 $-\hat{x} \text{mean}(a \odot \hat{x})$ 的意義是**把沿著 $\hat{x}$ 方向的分量扣掉**——因為 $\hat{x}$ 的長度已經被 r 固定住了，沿它自己的方向推是無效的。

Attention：含完整 softmax Jacobian

前向：

$$S = \frac{QK^\top}{\sqrt{d_{\text{head}}}}, \quad S_{ij} \leftarrow -\infty \text{ if } j > i, \quad A = \text{softmax}(S), \quad O = AV$$

除以 $\sqrt{d_{\text{head}}}$ 的理由： Q 、 K 各維度變異數約為 1，點積 d 個維度後變異數變成 d ，標準差 $\sqrt{d}$ 。 $d = 64$ 時分數會散到 ± 8 以上，softmax 吃到這種輸入會變得極度尖銳，梯度就消失了。

反向：

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial O} V^\top, \quad \frac{\partial L}{\partial V} = A^\top \frac{\partial L}{\partial O}$$

這裡沒有 cross-entropy 接在後面，所以要用**完整**的 softmax 反向（逐列獨立）：

$$\frac{\partial L}{\partial S_j} = A_j \left(\frac{\partial L}{\partial A_j} - \sum_i \frac{\partial L}{\partial A_i} A_i \right)$$

括號裡那個 $\sum$ 是「這一系列的加權平均」，每一列算一次就好。

$$\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{d}} \frac{\partial L}{\partial S} K, \quad \frac{\partial L}{\partial K} = \frac{1}{\sqrt{d}} \left(\frac{\partial L}{\partial S} \right)^\top Q$$

被遮罩的位置 $A_{ij} = 0$ ，公式裡的 $A_j(\dots)$ 自動讓梯度也是 0，**不需要另外處理**。

最後一棒：embedding 只做路由

embedding 的前向是「原封不動抄出來」，導數是 1。所以反向也是「原封不動接回去」——**它不對梯度做任何變換，只負責送回正確的列**。

$$\text{前向 gather} \iff \text{反向 scatter-add}$$

這個對稱不是設計出來的，是導數為 1 的必然結果。

一個實測現象：梯度會被放大

追蹤檔裡可以看到梯度範數逐層變化。從第 1 層傳到 embedding 時會突然放大約一個數量級。

原因是 **RMSNorm**。embedding 向量的 rms 約為 0.03：

$$\text{前向除以 } 0.03 \implies \text{放大約 } 33 \text{ 倍}$$

反向經過同一個除法時，梯度也跟著放大。（第二項 $-\hat{x} \text{mean}(a \odot \hat{x})$ 會抵消掉一部分，所以實測倍率比 $1/r$ 略小。）

怎麼確定推對了

不用 PyTorch 對答案，直接回到梯度的定義：

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w - \varepsilon)}{2\varepsilon}$$

```
python tests/test_layers.py
```

| 層 | 最大相對誤差 | |---|---|---| | Linear (無 bias) | 2.29×10^{-12} | PASS | | Linear (有 bias) | 2.03×10^{-12} | PASS | | RMSNorm | 2.13×10^{-7} | PASS | | Embedding (scatter-add) | 6.80×10^{-14} | PASS | | TiedOutputHead | 5.61×10^{-12} | PASS | | SwiGLU | 1.69×10^{-6} | PASS | | RoPE | 2.19×10^{-12} | PASS | | CausalSelfAttention | 3.32×10^{-7} | PASS | | Softmax + CrossEntropy | 7.50×10^{-8} | PASS |

用**中央差分**（兩邊各推一次）而不是單邊，因為誤差是 $O(\varepsilon^2)$ 而不是 $O(\varepsilon)$ 。

ε 也不能太小——`float32` 只有約 7 位有效數字，兩個相近的數相減會被捨入誤差吃掉。所以 `gradcheck` 預設用 `float64` 跑。

反向傳播不會修改任何權重

```
model.backward(dlogits)      # 只算出「該往哪修」，存進 dw / dE
opt.step()                   # 這一行才真的改動權重
```

`backward()` 跑完之後， E 一個數字都沒變。梯度存在另一個張量裡。

真正動手的是 optimizer，而且方向**相反**（往下坡走）、步伐只有 `lr` 那麼小。詳見 05-adamw。

權重是怎麼被改的

對應程式碼：`scratch/adamw.py`

分界線

```
model.backward(dlogits)      # 算出「該往哪修」 → 存進 dE、dw
opt.step()                   # 這一行才真的改動權重
```

反向傳播從不修改權重。這兩件事之間隔著一整張梯度表。

最直覺的版本：SGD

$$\theta \leftarrow \theta - \eta g$$

「減掉梯度乘上學習率」。

為什麼是減？梯度的定義是「往哪個方向動， L 會變大」：

$$g = \frac{\partial L}{\partial \theta}$$

我們要 L 變小，所以走反方向。這就是梯度下降的下降。

AdamW 做了什麼

$t \leftarrow t + 1$	
$\theta \leftarrow \theta - \eta \lambda \theta$	① decoupled weight decay（與梯度無關）
$m \leftarrow \beta_1 m + (1 - \beta_1) g$	② 一階動量：梯度的移動平均
$v \leftarrow \beta_2 v + (1 - \beta_2) g^2$	③ 二階動量：梯度平方的移動平均
$\hat{m} \leftarrow \frac{m}{1 - \beta_1^t}, \quad \hat{v} \leftarrow \frac{v}{1 - \beta_2^t}$	④ bias correction
$\theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$	⑤ 真正的更新

跟 SGD 的差別只在 ⑤： g 被換成了 $\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$ 。

bias correction 在做什麼： m 、 v 都從 0 出發，前幾步會系統性偏小。除以 $1 - \beta_1^t$ 剛好把這個偏差補回來（ t 大了之後這個係數趨近 1，就沒作用了）。

關鍵性質：位移量幾乎與梯度大小無關

看 ⑤ 的分子分母：分子是梯度的平均，分母是梯度大小的平均。量級相同，相除之後大約落在 ± 1 ，於是

$$|\Delta\theta| \approx \eta$$

第一步時這件事是精確成立的。代入 $m_0 = v_0 = 0$ ：

$$m_1 = (1 - \beta_1)g \implies \hat{m}_1 = \frac{m_1}{1 - \beta_1} = g$$

$$v_1 = (1 - \beta_2)g^2 \implies \hat{v}_1 = \frac{v_1}{1 - \beta_2} = g^2$$

$$\frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = \frac{g}{|g|} = \text{sign}(g) \implies \Delta\theta = \pm \eta$$

追蹤檔裡可以直接驗證：

	維度 1	維度 2	維度 3	維度 4
梯度 g	+0.244449	-0.286912	-0.239161	+0.236112
$\hat{m}/(\sqrt{\hat{v}} + \epsilon)$	+1.000000	-1.000000	-1.000000	+1.000000
位移 $\Delta\theta$	-0.000500	+0.000500	+0.000500	-0.000500
$ \Delta\theta /\eta$	1.0000	1.0000	1.0000	1.0000

(此處 $\eta = 5 \times 10^{-4}$ 。)

四個維度的梯度大小不同 ($0.236 \sim 0.287$)，位移卻一模一樣。

推論：總位移有上限

跑 T 步之後，任何一個參數的總位移上限大約是

$$\|\theta_T - \theta_0\| \lesssim T \cdot \bar{\eta}$$

實測驗算 (3000 步、cosine 從 3×10^{-3} 衰減)：

$$3000 \times 2.75 \times 10^{-4} \approx 0.825$$

而實測的平均位移落在 $0.63 \sim 0.86$ ——正好卡在這個上限。

差別只在方向有沒有一致：

	位移量	$\cos(\theta_0, \theta_T)$
高頻 token	接近上限	高（方向沒怎麼變）
中頻 token	接近上限	低（轉得最多）
從沒出現的 token	接近上限	中

位移量被 Adam 封頂，真正有差別的是「動得有沒有方向」。

高頻 token 出現在幾百萬個不同上下文，每個上下文想把它拉往不同方向，加起來互相抵消——所以它走得遠，但方向沒怎麼變。

decoupled weight decay

舊的 Adam 把 $\lambda\theta$ 加進梯度裡：

$$\theta \leftarrow \theta - \eta(g + \lambda\theta)$$

問題是這樣一來 weight decay 也會被 $\sqrt{\hat{v}}$ 正規化，強度變得跟梯度大小有關，很不直覺。

AdamW 把它拆出來獨立做（上面的 ①），強度固定是 $\eta\lambda$ 。這就是 **W** 的由來。

一個實務上的坑：`torch.optim.AdamW` 預設 $\lambda = 0.01$ ，而且會套用到**所有**參數，包括 embedding 和 norm 的 gain。

生產級訓練通常把這兩類排除——因為很少拿到梯度的列會被 weight decay 慢慢磨向 0。這份程式用 `no_decay` 參數控制，預設就排除了：

```
AdamW(model.params(), lr=..., no_decay=('.g', '.E'))
```

梯度裁剪

```
opt.clip_grad_norm(1.0)
```

把所有梯度串成一個超長向量，計算它的長度：

$$G = \sqrt{\sum_{\text{所有參數}} \|g\|^2}$$

若 $G > G_{\max}$ ，**整體**按比例縮小：

$$g \leftarrow g \cdot \frac{G_{\max}}{G + 10^{-6}}$$

注意是「整體」——各參數之間的相對比例不變。這跟逐元素裁剪（clip by value）不同，後者會扭曲梯度的方向。

用途是防止偶爾出現的異常大梯度（壞資料、數值不穩）一步把模型炸掉。

learning rate schedule

```
lr = cosine_lr(step, total, base_lr, warmup=100)
```

$$\eta(t) = \begin{cases} \eta_0 \cdot \frac{t}{T_{\text{warm}}}, & t < T_{\text{warm}} \\ \eta_0 \left[\rho + (1 - \rho) \cdot \frac{1 + \cos\left(\pi \frac{t - T_{\text{warm}}}{T - T_{\text{warm}}}\right)}{2} \right], & t \geq T_{\text{warm}} \end{cases}$$

其中 ρ 是最低點的比例（預設 0.1）。

- **warmup**：一開始權重是亂數，梯度方向不可靠，這時候用大 η 容易走歪。
- **cosine 衰減**：後期要小步微調，不然會在最低點附近震盪。

`traces/train_log.txt` 記了每一步的 η ，可以直接畫成曲線。

三個最常見的誤解

做簡報時最值得講的三件事。每一條都可以用這份專案的程式碼當場驗證。

① 「embedding 是資料，不是模型的一部分」

錯。它是權重，跟 `q_proj` 完全平起平坐。

會有這個誤解，是因為 `E` 長得像一張查詢表，直覺會把它歸類成「輸入」。

怎麼當場證明

```
python train.py --steps 1 --trace-steps 1
```

看印出來的參數清單：

```
model.embed.E          204,480    ← 在同一份清單裡
block0.attn.q_proj.W    16,384
block0.ffn.gate_proj.W  43,776
optimizer 拿到 20 個參數張量
```

再看追蹤檔裡兩者的更新：

```
位移 E[...]    -0.000500  +0.000500  +0.000500  -0.000500
位移 q_proj    +0.000500  +0.000500  -0.000500  -0.000500
```

同一個 optimizer、同一套規則、同樣的 $\pm lr$ 。

為什麼直覺會漏掉它

換個框架就通了——`E` 就是「第 0 層」：

	輸入	權重	輸出
第 0 層 (embedding)	one-hot (固定資料)	<code>E</code>	詞向量
第 1 層	詞向量	<code>w_q</code> , <code>w_k</code> , <code>w_v</code> , FFN	新的向量

第 1 層的「輸入」就是第 0 層的「輸出」。梯度傳到第 1 層的輸入時不會停在那裡，它繼續往下，變成 `E` 的梯度。

② 「反向傳播會修改權重」

錯。 `backward()` 跑完，權重一個數字都沒變。

```
model.backward(dlogits)      # 只算出「該往哪修」，存進另一個張量
opt.step()                   # 這一行才真的改
```

怎麼當場證明

追蹤檔第 04 節的最後，`E` 還是原值；到第 06 節 AdamW 那裡才變。

而且方向是相反的，步伐小得多：

更新前 <code>E[...]</code>	-0.01533508	
梯度 <code>g</code>	+0.24444881	
若直接「加上梯度」	+0.22911373	← 錯誤的直覺
AdamW 實際結果	-0.01583501	← 差了 489 倍

兩層都要注意：

1. 梯度是先彼此相加存進梯度表，不是加到 `E` 上
2. `E` 真的要改的時候是減，而且要乘上很小的 `lr`

③ 「token id 是離散的，所以沒辦法微分」

混淆了兩個東西。不對 id 微分——它是資料不是參數。

	是什麼	離散/連續	微分嗎
<code>id = 4</code>	資料	離散	不用，也不能
<code>E[4][0] = -0.0153</code>	參數	連續實數	對它微分

梯度問的是：

$$\frac{\partial L}{\partial E_{4,0}} = \lim_{\varepsilon \rightarrow 0} \frac{L(E_{4,0} + \varepsilon) - L(E_{4,0} - \varepsilon)}{2\varepsilon}$$

跟 id 是不是離散完全無關。

怎麼當場證明

$$f(a, b, c) = a + b \quad (c \text{ 沒參與})$$

$$\frac{\partial f}{\partial a} = 1, \quad \frac{\partial f}{\partial c} = 0$$

$\partial f / \partial c = 0$ 不是「不能微分」，是「算出來就是 0」。

寫成矩陣就完全沒有特殊性了：

$$X = O E \quad (O \text{ 是常數 one-hot 矩陣, } E \text{ 是變數})$$

$$\frac{\partial L}{\partial E} = O^\top G \quad \leftarrow \text{課本上最普通的矩陣微分}$$

$O^\top G$ 展開就是

$$\frac{\partial L}{\partial E_v} = \sum_{t: \text{id}_t=v} G_t$$

剛好就是「同一列的相加、沒出現的是 0」—— **scatter-add 不是特殊規則，是矩陣乘法自然的結果。**

有限差分可以直接驗證 ($\varepsilon = 10^{-3}$)：

格子	$\frac{L_+ - L_-}{2\varepsilon}$	公式	
$E_{2,0}$ (在)	0.246048	0.246000	查兩次，斜率加倍
$E_{1,0}$ (狗)	0.000000	0.000000	沒被查到

「狗」那一列 $L(w + \varepsilon)$ 和 $L(w - \varepsilon)$ **完全相等**—— 那個數字沒參與計算，動它 L 不會變。

延伸：離散在哪裡才真的會咬人

情境	可微嗎
訓練 (teacher forcing)	可微，id 是給定的常數
生成 (argmax / 取樣)	不可微 ，梯度在這裡斷掉

第二種才是真麻煩——RLHF / GRPO 需要 policy gradient 就是為了繞過它。

附：一個 bonus 誤解

「embedding 的梯度是稀疏的」—— 在有 weight tying 的模型上**不成立**。

	來源	有幾列非零
輸入側 (scatter-add)	272 / 3195	← 稀疏
輸出側 (weight tying)	3195 / 3195	← 稠密
合計	3195 / 3195	

因為 `E` 同時是輸出層的權重，softmax 要對全部 token 算分數，**每一列每一步都會被碰到**——連完全沒出現在這批資料裡的 token 也一樣。

想親眼看差別：`TinyLM(..., tie_weights=False)` 跑一次，非零列就會剩下 272。

權重與 tokenizer 是綁死的

「別人拿到這些權重做 continual pre-training，tokenizer 也要一樣才行，對嗎？」

對，而且比多數人以為的更嚴格。

不只詞表大小要一樣——**每一個 token 對應到哪個 id，都必須完全相同。**

為什麼

E 的第 v 列，存的是「訓練期間 $id = v$ 代表的那個 token」的向量。

訓練時 $id\ 4 = \text{'的'}$ $\rightarrow E[4]$ 學成了「的」的向量

換 tokenizer 後 $id\ 4 = \text{'，'}$ $\rightarrow$ 模型拿「的」的向量去理解「，」

而且因為 weight tying， E 同時是輸出層——**輸入查表跟輸出分類會一起錯亂。**

實測：錯得有多離譜

```
python tests/test_tokenizer_binding.py
```

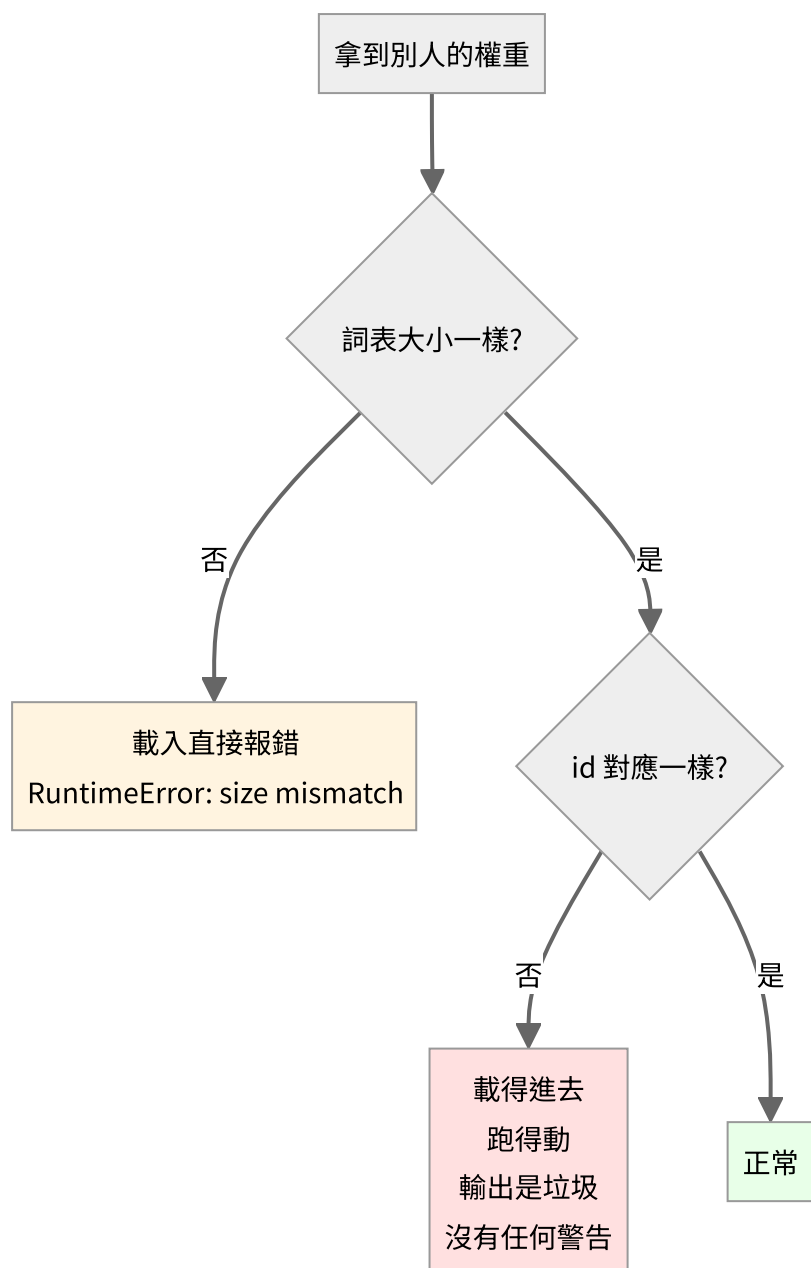
同一個模型、同一句話，只是把 id 對應打亂（模擬「同語料但不同種子重訓的 tokenizer」）：

	loss	perplexity（等效於在幾個選項中猜）
正確的 tokenizer	3.61	37
id 對應被打亂	12.87	389,971
完全沒訓練過的模型	—	6,400（= 詞表大小）

用錯 tokenizer 比完全沒訓練過的模型還糟 61 倍。

為什麼會比隨機還差？因為隨機模型是均勻分布，至少「不確定」；而載錯 tokenizer 的模型會**很有自信地猜錯**——把高機率押在錯誤的 token 上。

三種情況，危險程度不同



| 情況 | 會怎樣 | 危險嗎 | |---|---|---| | 詞表大小不同 | `RuntimeError: size mismatch` | **不危險**——馬上就知道出事了 | | 大小相同、id 對應不同 | 安然載入、正常執行、輸出垃圾 | **非常危險**——沒有任何提示 |

第二種才是真正的坑。你會以為模型壞了、資料有問題、超參數不對，花好幾天調參，實際上只是 tokenizer 拿錯了。

防呆：把 tokenizer 的指紋存進 checkpoint

這份專案的做法：

```
# tokenizer/bpe.py
def fingerprint(self):
    s = '|'.join(self.itos) + '|'.join('%s>%s' % m for m in self.merges)
    return hashlib.sha256(s.encode('utf-8')).hexdigest()[:16]
```

指紋涵蓋 **id 的順序** (`itos` 是有序的)，所以順序一變就會被抓到。

存檔時一起寫進去，載入時比對：

```
model.save('out/model.pt', tokenizer_fingerprint=tok.fingerprint())
model.load('out/model.pt', tokenizer_fingerprint=tok.fingerprint())
```

拿錯就直接擋下來：

```
ValueError: tokenizer 不符！
權重存檔時用的 : dcb6ef062b94a20e
現在載入的      : deadbeefdeadbeef
詞表大小可能一樣，但 id 對應不同——模型會照跑，輸出卻是垃圾。
要強制載入請傳 strict_tokenizer=False。
```

這是 20 行程式碼就能省下好幾天除錯時間的那種投資。

如果真的需要改 tokenizer

可以：擴充 (append)

保留原有的 id `0..V-1` 完全不動，只在後面接新的列：

```
p[:old_vocab].copy_(old_E)          # 原有 id 原封不動
p[old_vocab:] = old_E.mean(0)        # 新列用既有向量的平均值起步
```

實測 (詞表 6,400 → 6,450)：

```
同一句話的 loss = 3.6130    (原本 3.6129，差 0.0000)
```

幾乎沒有影響，因為原有的 id 對應沒被動到。這就是做語言擴充、加領域詞彙時的標準做法。

新列的初始化有幾種常見選擇：

做法	說明
既有向量的平均	最簡單，起點中性
小亂數	跟從頭訓練一致
子詞平均	新 token 是 機器學習，就用 機、器、學、習 四個舊向量的平均

第三種通常最好——新 token 一開始就落在語意上合理的位置。

不行：重建（rebuild）

用新語料重訓一個 tokenizer，即使詞表大小相同、甚至詞彙集合相同，只要 id 的分配順序不同，權重就報廢了。

BPE 的 id 順序取決於合併的順序，而合併順序取決於語料的統計。換語料、換種子、甚至換一個實作的 tie-breaking 規則，順序就會不一樣。

折衷：詞表移植（tokenizer transplant）

如果非換不可，可以做「字串比對搬家」：

對新詞表的每個 token：

如果舊詞表裡有同樣的字串 → 把舊向量搬過來

否則 → 用子詞平均初始化

能救回一部分，但一定會掉品質，需要再訓練一段時間補回來。業界做跨語言適配時常用這招，但那是不得已，不是首選。

一句話

權重跟 tokenizer 是一組的，發布時要一起發布，換模型時要一起換。

只能往後擴充，不能重建。

這也是為什麼 HuggingFace 上的模型 repo 一定會把 `tokenizer.json` 跟 `model.safetensors` 放在同一個目錄——它們不是兩個獨立的東西。

RMSNorm

對應程式碼：`scratch/rmsnorm.py`

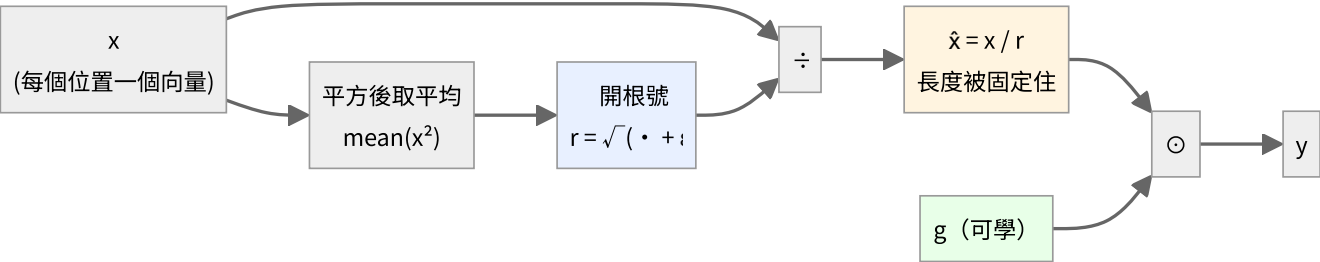
每一層的入口都會先過它。整個模型裡它的參數量最少（每層只有 d 個），但影響最大——它決定了進入 Attention 和 FFN 的數值尺度。

前向

$$r = \sqrt{\frac{1}{n} \sum_{k=1}^n x_k^2 + \varepsilon}$$
$$\hat{x} = \frac{x}{r}$$
$$y = g \odot \hat{x}$$

三個步驟，三個角色：

符號	是什麼	可學嗎
r	這一系列數字的均方根（root-mean-square）	否，算出來的
$\hat{x}$	正規化後的向量，長度被拉到約 $\sqrt{n}$	否
g	每個維度一個的縮放係數	是，初始值全為 1
ε	防止除以零，預設 10^{-5}	否，固定常數



沿哪一維做正規化很重要：是沿最後一維（特徵維度），所以每個位置各自正規化，位置與位置之間互不影響。

跟 LayerNorm 的差別

LayerNorm: $y = g \odot \frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + b$

$$\text{RMSNorm: } y = g \odot \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}}$$

RMSNorm 少了兩樣東西：

1. 不減平均數 μ
2. 沒有偏置 b

少一次減法、少一組參數，實測效果幾乎一樣。所以現在的 LLM（LLaMA、Mistral、Qwen、minimind）幾乎都用 RMSNorm。

實際數值

從 `traces/sample/train_step_0001.txt` 抓的第一層 `norm1`：

	前 4 個維度
x （輸入）	+0.01647 - 0.04764 + 0.02504 + 0.00472
r	0.019711
$\hat{x}$ （正規化後）	+0.83548 - 2.41710 + 1.27034 + 0.23929
g （初始值）	+1.00000 + 1.00000 + 1.00000 + 1.00000
y （輸出）	+0.83548 - 2.41710 + 1.27034 + 0.23929

驗算第一個維度：

$$\hat{x}_1 = \frac{0.01647}{0.019711} = 0.83557 \approx 0.83548$$

（差在四捨五入。）

注意這個放大倍率：

$$\frac{1}{r} = \frac{1}{0.019711} = 50.7$$

embedding 出來的向量很小（每個元素約 0.02），RMSNorm 把它們放大了 50 倍。這不是 bug，正是它的用途——把不同來源、不同尺度的向量統一到同一個工作區間。

反向（逐步推導）

設 n 是維度數， $a_i = \frac{\partial L}{\partial y_i} g_i$ 。

第一步： r 對 x 的偏導

$$r = \left(\frac{1}{n} \sum_k x_k^2 + \varepsilon \right)^{1/2}$$

用連鎖律：

$$\frac{\partial r}{\partial x_j} = \frac{1}{2} \left(\frac{1}{n} \sum_k x_k^2 + \varepsilon \right)^{-1/2} \cdot \frac{2x_j}{n} = \frac{x_j}{n r}$$

第二步：y 對 x 的偏導

這裡是關鍵—— y_i 透過兩條路依賴 x_j ：

1. 分子裡的 x_i 本身（只有 $i = j$ 時）
2. 分母 r 裡面也含有 x_j （每個 j 都有）

$$y_i = g_i \frac{x_i}{r}$$

用商法則：

$$\frac{\partial y_i}{\partial x_j} = g_i \left[\frac{\delta_{ij}}{r} - \frac{x_i}{r^2} \cdot \frac{\partial r}{\partial x_j} \right] = g_i \left[\frac{\delta_{ij}}{r} - \frac{x_i x_j}{n r^3} \right]$$

其中 δ_{ij} 是 Kronecker delta（ $i = j$ 時為 1，否則為 0）。

第三步：連鎖律加總

$$\frac{\partial L}{\partial x_j} = \sum_i \frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial x_j} = \frac{a_j}{r} - \frac{x_j}{n r^3} \sum_i a_i x_i$$

第四步：化簡

把 $x = \hat{x} r$ 代入第二項：

$$\frac{x_j}{n r^3} \sum_i a_i x_i = \frac{\hat{x}_j r}{n r^3} \cdot r \sum_i a_i \hat{x}_i = \frac{\hat{x}_j}{r} \cdot \frac{1}{n} \sum_i a_i \hat{x}_i$$

得到最後要寫進程式的形式：

$$\boxed{\frac{\partial L}{\partial x} = \frac{1}{r} \left(a - \hat{x} \cdot \text{mean}(a \odot \hat{x}) \right), \quad a = \frac{\partial L}{\partial y} \odot g}$$

對 g 的梯度

單純得多，因為 $y = g \odot \hat{x}$ 裡 g 只出現一次：

$$\frac{\partial L}{\partial g} = \sum_{\text{所有位置}} \frac{\partial L}{\partial y} \odot \hat{x}$$

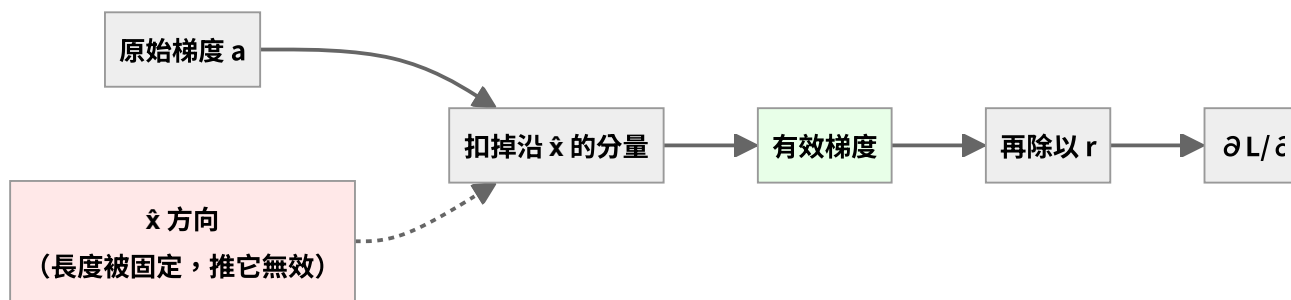
第二項在做什麼

$$-\hat{x} \cdot \text{mean}(a \odot \hat{x})$$

它的作用是把梯度中沿著 $\hat{x}$ 方向的分量扣掉。

理由： $\hat{x}$ 的長度已經被 r 固定住了 ($\|\hat{x}\| = \sqrt{n}$)。沿著它自己的方向推，只會改變長度——而長度馬上又會被正規化消掉，所以那個方向的梯度是無效的，扣掉才正確。

幾何上，這是把梯度投影到與 $\hat{x}$ 垂直的超平面上。



一個實測現象：梯度會被放大

前向除以 $r = 0.0197$ ，反向經過同一個除法時，梯度也會被放大差不多的倍數。

這就是為什麼梯度從第 1 層傳到 embedding 時會突然大一個數量級。

實測倍率會比 $1/r = 50.7$ 略小，因為第二項扣掉了一部分。

驗證

`tests/test_layers.py` 用有限差分驗證這一層：

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w - \varepsilon)}{2\varepsilon}$$

結果：最大相對誤差 2.13×10^{-7} ，**PASS**。

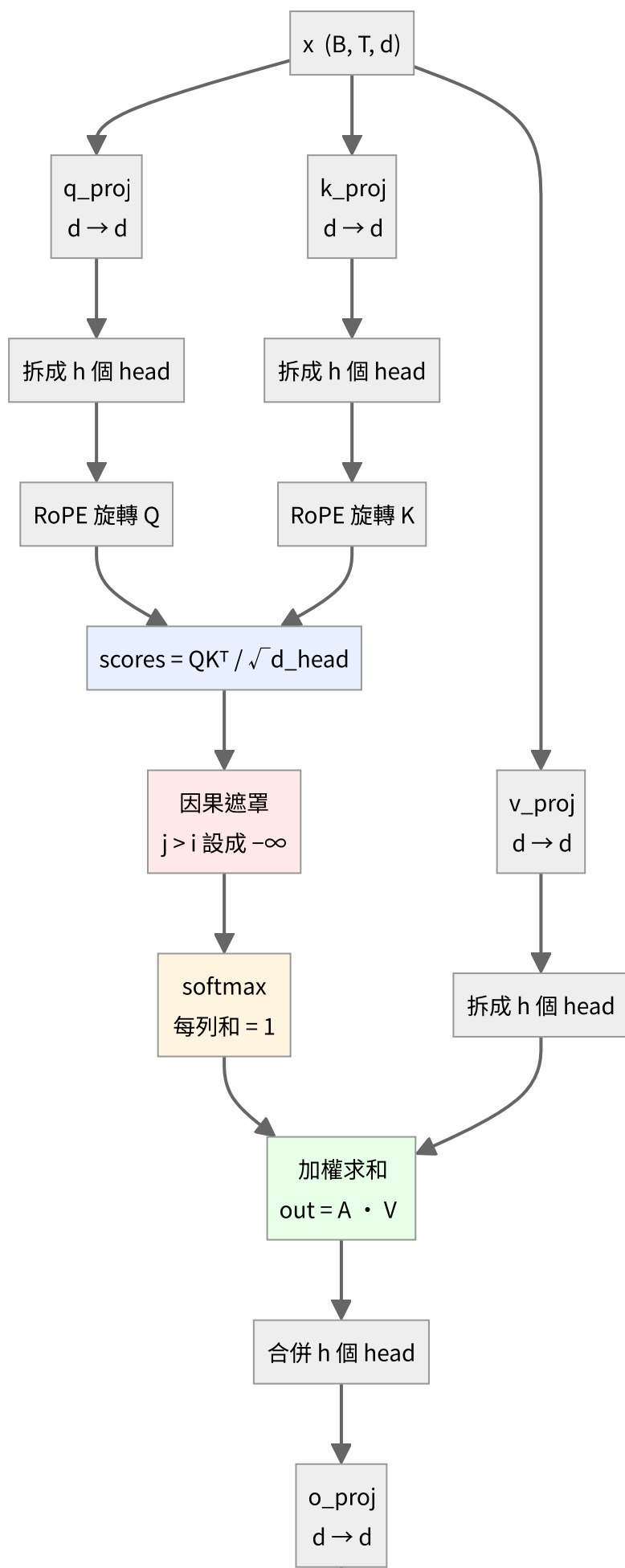
(這一層的誤差比 Linear 的 10^{-12} 大，是因為它含有開根號和除法，浮點運算的累積誤差比較多。 10^{-7} 對 `float64` 來說仍在合理範圍。)

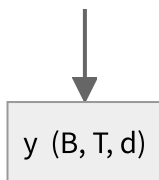
Attention 的結構

對應程式碼：[scratch/attention.py](#)

整個模型裡**唯一讓不同位置互相看得到的地方**。FFN 是每個位置各做各的，只有 Attention 會橫向交換資訊。

全貌





注意 **V 不做 RoPE**——位置資訊只需要影響「誰該注意誰」（分數），不需要影響「注意到之後拿到什麼內容」。

一、三條投影

$$Q = xW_Q^\top, \quad K = xW_K^\top, \quad V = xW_V^\top$$

同一個 x 走了三條獨立的線性層，產生三個角色：

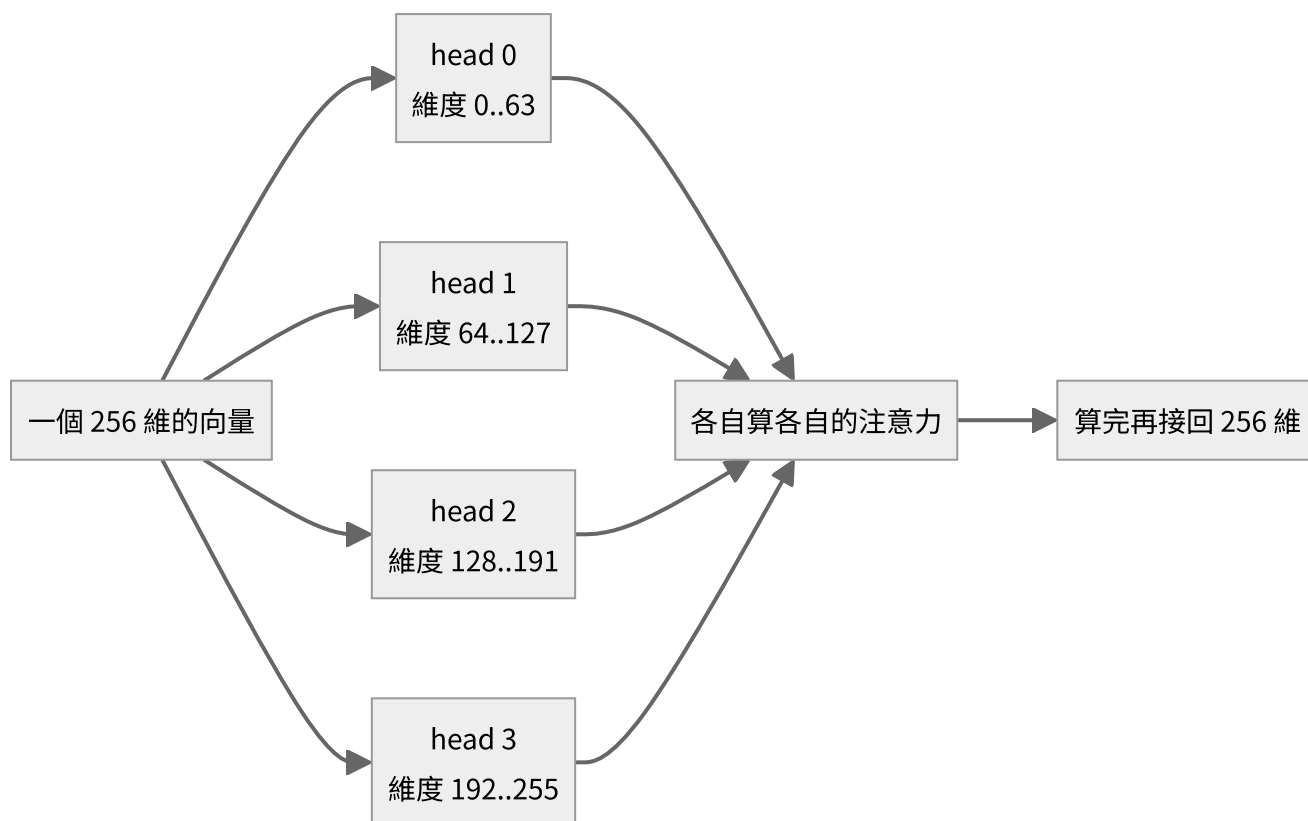
	名稱	直覺
Q	Query，查詢	「我在找什麼」
K	Key，鍵	「我是什麼」
V	Value，值	「我能提供什麼內容」

因為 x 分岔成三路，**反向時三份梯度要相加**：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial x}\Big|_Q + \frac{\partial L}{\partial x}\Big|_K + \frac{\partial L}{\partial x}\Big|_V$$

二、拆成多個 head

$$(B, T, d) \longrightarrow (B, h, T, d_{\text{head}}), \quad d_{\text{head}} = \frac{d}{h}$$



不是複製四份，是把 256 維切成四段，每段獨立算注意力。這樣參數量完全沒增加，但模型可以同時關注不同類型的關係（例如一個 head 管語法、一個管指代）。

三、注意力分數

$$S = \frac{QK^{\top}}{\sqrt{d_{\text{head}}}}$$

S_{ij} 的意思是「位置 i 對位置 j 的關注程度」，本質是兩個向量的點積。

為什麼要除以 $\sqrt{d_{\text{head}}}$

假設 Q 、 K 各維度獨立、變異數約為 1，那麼點積

$$S_{ij} = \sum_{k=1}^{d_{\text{head}}} q_{ik} k_{jk}$$

是 d_{head} 個獨立項的和，變異數變成 d_{head} ，標準差 $\sqrt{d_{\text{head}}}$ 。

$d_{\text{head}} = 64$ 時，分數會散到 ± 8 以上。softmax 吃到這種輸入會變得**極度尖銳**（幾乎變成 one-hot），而 softmax 在飽和區的梯度趨近於零——**梯度就消失了**。

除以 $\sqrt{d_{\text{head}}}$ 把變異數拉回 1，softmax 才處在有梯度的區間。

實測本專案的設定： $d_{\text{head}} = 64$ ， $\text{scale} = 1/\sqrt{64} = 0.125$ 。

四、因果遮罩

語言模型只能用「已經看過的字」預測下一個字。所以位置 i 只能注意 $j \leq i$ ：

$$S_{ij} \leftarrow -\infty \quad \text{if } j > i$$

$\exp(-\infty) = 0$ ，所以那些位置的注意力權重變成 0，等於不存在。

遮罩矩陣長這樣（✓ 可以看，空白被擋住）：

	j=0	j=1	j=2	j=3	j=4
i=0	✓				
i=1	✓	✓			
i=2	✓	✓	✓		
i=3	✓	✓	✓	✓	
i=4	✓	✓	✓	✓	✓

下三角。這也是為什麼一次前向就能同時訓練 T 個位置的預測——每個位置看到的都是「只到自己為止」的資訊，互不干擾。

五、softmax

$$A = \text{softmax}(S), \quad A_{ij} = \frac{\exp(S_{ij})}{\sum_k \exp(S_{ik})}$$

沿**最後一維**做，所以每一列加起來等於 1：

$$\sum_j A_{ij} = 1 \quad \text{對每個 } i$$

實際跑出來的注意力矩陣（`traces/sample/train_step_0001.txt`，第 0 個 head，訓練第 1 步）：

	j=0	j=1	j=2	j=3	j=4	列和
i=0	1.00000					1.0
i=1	0.43477	0.56523				1.0
i=2	0.35364	0.28259	0.36377			1.0
i=3	0.21434	0.32812	0.22678	0.23077		1.0
i=4	0.23005	0.15862	0.16002	0.20928	0.24203	1.0

三件事一眼可見：

1. 上三角全是空的——因果遮罩生效了
 2. 每列和都是 1.0——softmax 正確
 3. 權重相當平均 ($i = 4$ 那列都在 $0.16 \sim 0.24$ 之間) ——因為這是**訓練第 1 步**，權重還是亂數，模型還不知道該注意誰。訓練久了會出現明顯的尖峰。
-

六、加權求和與合併

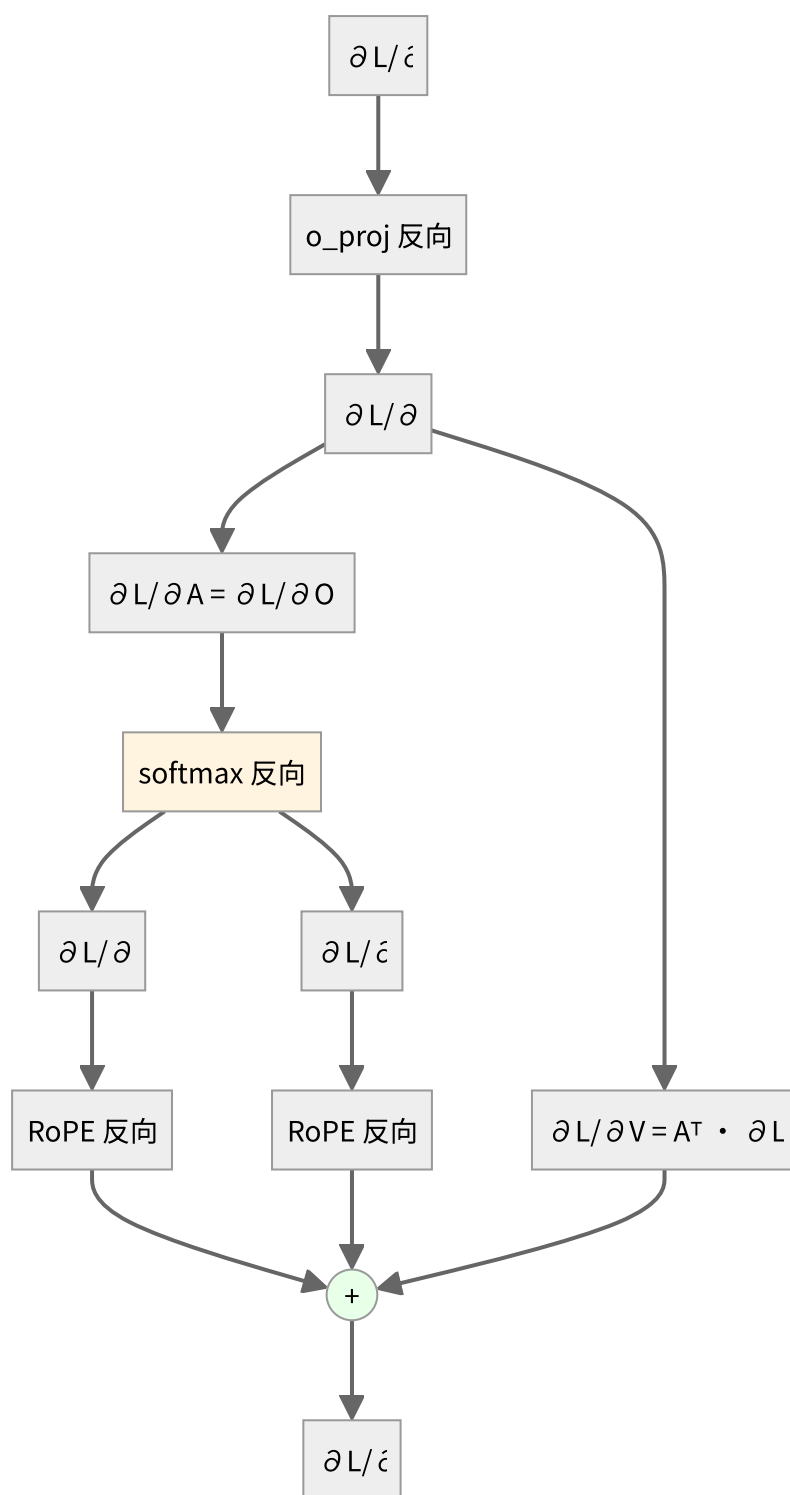
$$O = AV, \quad O_i = \sum_j A_{ij} V_j$$

每個位置的輸出，是**所有它看得到的位置的 V 的加權平均**，權重就是注意力。

$$(B, h, T, d_{\text{head}}) \longrightarrow (B, T, d) \xrightarrow{W_O} y$$

`o_proj` 的作用是把各個 head 的結果**混合**起來——沒有它，各 head 算完就直接拼接，彼此之間永遠不會互動。

反向



對 A 與 V

$O = AV$ 是矩陣乘法，套用標準結果：

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial O} V^{\top}, \quad \frac{\partial L}{\partial V} = A^{\top} \frac{\partial L}{\partial O}$$

(又是「點積對一邊微分得到另一邊」。)

softmax 的完整反向

這裡**沒有** cross-entropy 接在後面，所以不能用 $p - y$ 那個簡化，必須用完整形式。對每一列 i 獨立：

$$\frac{\partial A_{ij}}{\partial S_{ik}} = A_{ij} (\delta_{jk} - A_{ik})$$

$$\boxed{\frac{\partial L}{\partial S_{ij}} = A_{ij} \left(\frac{\partial L}{\partial A_{ij}} - \sum_k \frac{\partial L}{\partial A_{ik}} A_{ik} \right)}$$

括號裡那個 $\sum$ 是**這一系列的加權平均**，每列算一次就好，不需要真的建出 $T \times T$ 的 Jacobian。

遮罩不用特別處理：被遮住的位置 $A_{ij} = 0$ ，公式最前面的 A_{ij} 自動讓那些梯度也是 0。

對 Q 與 K

$$\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{d_{\text{head}}}} \frac{\partial L}{\partial S} K, \quad \frac{\partial L}{\partial K} = \frac{1}{\sqrt{d_{\text{head}}}} \left(\frac{\partial L}{\partial S} \right)^\top Q$$

記憶體：為什麼 Attention 是 $O(T^2)$

分數矩陣的形狀是

$$S \in \mathbb{R}^{B \times h \times T \times T}$$

以 $B = 64$ 、 $h = 4$ 、 $T = 128$ 為例：

$$64 \times 4 \times 128 \times 128 = 4.19 \times 10^6$$

序列長度加倍，這個張量會變成**四倍**。而且訓練時 A 要存下來給反向用。

這就是長 context 昂貴的根本原因，也是 FlashAttention 這類技術要解決的問題（它的作法是分塊計算、不把完整的 S 物化出來）。

驗證

`tests/test_layers.py` 對這一層的有限差分驗證：最大相對誤差 3.32×10^{-7} ，**PASS**。

含 softmax Jacobian、因果遮罩、RoPE 反向、三路梯度相加，全部推對。

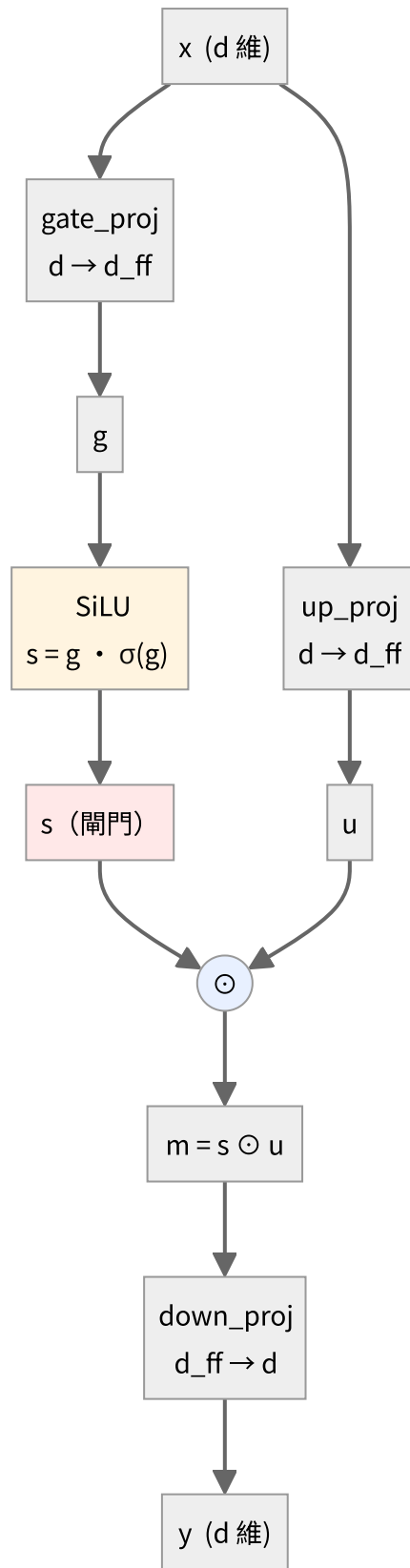
FFN 的結構 (SwiGLU)

對應程式碼：[scratch/swiglu.py](#)

Attention 負責**橫向**（位置之間交換資訊），FFN 負責**縱向**（每個位置各自加工）。FFN 對第 7 個位置做的事，跟它對第 13 個位置做的事完全獨立。

而且它是每一層裡**參數量最大**的部分——通常佔三分之二以上。

全貌



$$g = xW_{\text{gate}}^{\top}, \quad u = xW_{\text{up}}^{\top}, \quad s = \text{SiLU}(g), \quad m = s \odot u, \quad y = mW_{\text{down}}^{\top}$$

寫成一行：

$$y = \left(\text{SiLU}(xW_{\text{gate}}^{\top}) \odot xW_{\text{up}}^{\top} \right) W_{\text{down}}^{\top}$$

跟原始 Transformer 的 FFN 比較

原始 (2017) : $y = \text{ReLU}(xW_1 + b_1)W_2 + b_2$

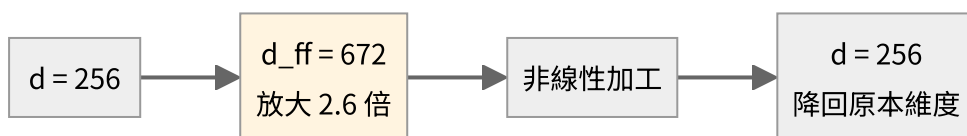
SwiGLU: $y = (\text{SiLU}(xW_{\text{gate}}) \odot xW_{\text{up}})W_{\text{down}}$

	原始	SwiGLU
矩陣數	2	3
非線性	ReLU	SiLU (平滑, 處處可微)
閘門機制	無	有 (多出來的 up 那一路)
bias	有	無

多出來的那一路 u 的作用是**開關**： s 決定 u 的每個維度要放行多少。

因為多了一個矩陣，為了維持總參數量相當， d_{ff} 通常取 $\frac{8}{3}d$ 而不是傳統的 $4d$ 。

為什麼要先升維再降回來



在高維空間裡，資料更容易被非線性「切開」。先升維、加工、再壓回來——這是 FFN 的標準套路。

參數量因此很可觀：

$$|\theta_{\text{FFN}}| = 3 \times d \times d_{\text{ff}} = 3 \times 256 \times 672 = 516,096$$

$$|\theta_{\text{Attention}}| = 4 \times d^2 = 4 \times 256^2 = 262,144$$

FFN 大約是 Attention 的兩倍。**一層裡三分之二的參數都在 FFN。**

SiLU 這個非線性

$$\text{SiLU}(z) = z \sigma(z) = \frac{z}{1 + e^{-z}}$$

(也叫 Swish。)

z	$\sigma(z)$	SiLU(z)
-3	0.047	-0.142
-1	0.269	-0.269
0	0.500	0.000
1	0.731	0.731
3	0.953	2.858

跟 ReLU 比：

- **負數區不是直接砍成 0**，而是留下一點負值（最小值約 -0.278 ）。ReLU 的「死神經元」問題（一旦進入負區梯度永遠是 0）在這裡不存在。
- **處處可微**，沒有 ReLU 在 $z = 0$ 的折點。

導數（用 $\sigma' = \sigma(1 - \sigma)$ 推）：

$$\text{SiLU}'(z) = \sigma(z) + z \sigma(z)(1 - \sigma(z)) = \sigma(z) \left(1 + z(1 - \sigma(z)) \right)$$

閘門在做什麼（用真實數值看）

從 `traces/sample/train_step_0001.txt` 抓的第一層 FFN，前 4 個維度：

維度	g (gate)	$s = \text{SiLU}(g)$	u (up)	$m = s \odot u$
1	-1.00882	-0.26957	-0.59599	+0.16066
2	-0.32442	-0.13613	+0.28378	-0.03863
3	+0.33926	+0.19813	-0.03482	-0.00690
4	-1.21415	-0.27800	-0.69361	+0.19282

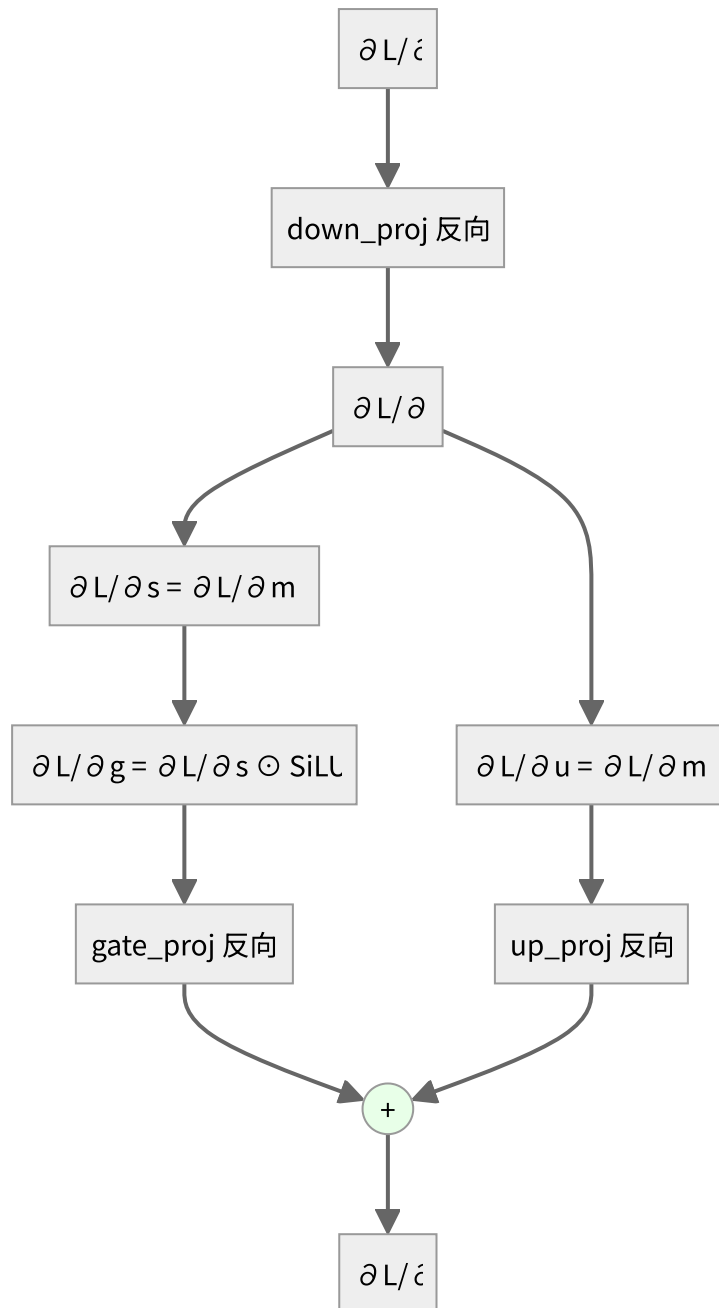
驗算第 1 個維度：

$$\text{SiLU}(-1.00882) = -1.00882 \times \sigma(-1.00882) = -1.00882 \times 0.2672 = -0.2696 \checkmark$$

$$m_1 = s_1 \times u_1 = (-0.26957) \times (-0.59599) = +0.16066 \checkmark$$

看第 3 個維度： $u_3 = -0.03482$ 本身就很小，乘上 $s_3 = 0.198$ 之後變成 -0.0069 ——**幾乎被關掉了**。這就是 gating：即使 u 那一路算出了東西， s 可以決定讓它過多少。

反向



逐條寫出來：

$$\frac{\partial L}{\partial m} = \text{down_proj.backward}\left(\frac{\partial L}{\partial y}\right)$$

$m = s \odot u$ 是逐元素相乘，對一邊微分得到另一邊：

$$\frac{\partial L}{\partial s} = \frac{\partial L}{\partial m} \odot u, \quad \frac{\partial L}{\partial u} = \frac{\partial L}{\partial m} \odot s$$

再過 SiLU 的導數：

$$\frac{\partial L}{\partial g} = \frac{\partial L}{\partial s} \odot \text{SiLU}'(g)$$

最後兩路回到 x ，**梯度相加**：

$$\frac{\partial L}{\partial x} = \text{gate_proj.backward}\left(\frac{\partial L}{\partial g}\right) + \text{up_proj.backward}\left(\frac{\partial L}{\partial u}\right)$$

x 同時餵給 gate 和 up 兩條路，所以反向要把兩份加起來——**前向分岔，反向相加**，這條通則在整份程式碼裡反覆出現。

記憶體：FFN 是訓練時的活化值大戶

反向需要用到 g 、 s 、 u ，所以前向時全都要存下來。每一個的形狀都是 (B, T, d_{ff}) 。

以 $B = 64$ 、 $T = 128$ 、 $d_{\text{ff}} = 672$ 、bf16 為例，單一個張量：

$$64 \times 128 \times 672 \times 2 \text{ bytes} = 11 \text{ MB}$$

一層要存三個 (g 、 s 、 u) 約 33 MB，四層就是 132 MB。

這就是「訓練比推論吃記憶體」的主因之一——**推論不用存這些，算完就丟**。

驗證

`tests/test_layers.py` 對這一層的有限差分驗證：最大相對誤差 1.69×10^{-6} ，**PASS**。

（這是九層裡誤差最大的一個，因為 SiLU 的導數含有 $\sigma(1 - \sigma)$ ，連續兩次浮點運算的誤差會累積。 10^{-6} 仍在合理範圍。）

完整執行紀錄

從乾淨狀態跑完整條流程，每個階段都是「指令 → 程式碼在做什麼 → 實際輸出」三段式。

這份文件怎麼讀

追蹤檔的資料量很大——單一個 `train_step_0001.txt` 就有 **926 行 / 59 KB**。全部貼出來沒有意義，所以這裡採三層呈現：

層	做什麼	在哪
摘要	一張表說明產出了什麼檔案、多大、要看什麼	產出的檔案
骨架	長檔案的章節目錄與行號，知道去哪裡找	怎麼讀 926 行的追蹤檔
精選	5 個關鍵片段，配上對應的程式碼	下面每個階段

原則：這份文件負責告訴你「該看哪幾行」，不負責取代那些檔案。

環境

GPU	NVIDIA RTX PRO 6000 Blackwell (96 GB)
Python	3.12, torch 2.9.1+cu130
語料	<code>data/corpus_zhtw.jsonl</code> 前 20,000 行
總耗時	梯度驗證 3 秒 + BPE 訓練 12 秒 + 模型訓練 79 秒

```
rm -rf out traces      # 從乾淨狀態開始
```

階段 1：驗證所有手推的公式

```
python tests/test_layers.py
```

程式碼在做什麼

不用 PyTorch 的 autograd 對答案，而是回到梯度的**定義**直接量：

$$\frac{\partial L}{\partial w} \approx \frac{L(w + \varepsilon) - L(w - \varepsilon)}{2\varepsilon}$$

```
# scratch/gradcheck.py (節錄)
y0 = layer.forward(x)
dy = torch.randn(y0.shape)      # 隨機的上游梯度
dx = layer.backward(dy)          # 手推公式算出來的

flat[i] = orig + eps; lp = float((layer.forward(x) * dy).sum())
flat[i] = orig - eps; lm = float((layer.forward(x) * dy).sum())
num = (lp - lm) / (2 * eps)      # 有限差分算出來的
rel = abs(num - float(dx.reshape(-1)[i])) / ...
```

人工造一個純量 $\text{loss } L = \sum(y \odot dy)$ ，這樣 $\partial L / \partial y$ 剛好等於 dy ，就能用任意的 dy 去測 backward 對不對。

用**中央差分**而不是單邊，因為誤差是 $O(\epsilon^2)$ 而不是 $O(\epsilon)$ 。整個檢查跑在 `float64` 上，避免 `float32` 的捨入誤差蓋過訊號。

實際輸出

梯度檢查（有限差分 vs 手推公式）

```
=====
Linear 無bias          最大相對誤差 2.29e-12  PASS
Linear 有bias          最大相對誤差 2.03e-12  PASS
RMSNorm                最大相對誤差 2.13e-07  PASS
Embedding (scatter-add) 最大相對誤差 6.80e-14  PASS
TiedOutputHead         最大相對誤差 5.61e-12  PASS
SwiGLU                 最大相對誤差 1.69e-06  PASS
rope                   最大相對誤差 2.19e-12  PASS
Attention              最大相對誤差 3.32e-07  PASS
Softmax+CrossEntropy   最大相對誤差 7.50e-08  PASS
=====
最差相對誤差 1.69e-06  -> 全部通過
```

誤差大小的差異是有意義的：純矩陣運算的層（Linear、Embedding、RoPE）在 10^{-12} 等級，含開根號、除法、 $\sigma(1 - \sigma)$ 的層（RMSNorm、SwiGLU）在 $10^{-6} \sim 10^{-7}$ ，因為浮點誤差會累積。

階段 2：訓練 BPE tokenizer

```
python train.py --tokenizer bpe --vocab 6400 ...
```

程式碼在做什麼

$$|\mathcal{V}| = |\mathcal{V}_0| + M$$

反覆合併最高頻的相鄰配對，每合併一次詞表 +1，到目標大小就停。

```
# tokenizer/bpe.py (節錄)
for step in range(n_merges):
    pair, cnt = max(pair_cnt.items(), key=lambda kv: kv[1]) # 最高頻的配對
    new_sym = pair[0] + pair[1]
    for i in pair_where[pair]: # 只更新受影響的詞塊
        deindex_word(i)
        ...把 pair 換成 new_sym...
        index_word(i)
    tk.merges.append(pair)
    tk.itos.append(new_sym) # 詞表 +1
```

`pair_where` 記錄「哪些詞塊含有這個配對」，合併時只更新受影響的那些，不用每輪重掃整份語料——否則 1,173 次合併會跑很久。

實際輸出

```
詞塊 1,826,987 個（相異 599,629 個），字元 5,415,807 個
起始詞表 5,227（4 特殊 + 5,223 字元）
目標 6,400 → 需要合併 1,173 次

合併 #1      '可' + '以' -> '可以'（23,763 次）  詞表 5,228
合併 #1000   '生' + '產' -> '生產'（393 次）   詞表 6,227

完成，最終詞表 6,400
```

$5,227 + 1,173 = 6,400$ 。這就是「詞表 6400 哪來的」的答案。

注意合併次數一路遞減： $23,763 \rightarrow 393$ 。這給了一個實用判斷——如果最後幾次合併降到個位數，代表詞表開太大了。

完整的合併紀錄：`traces/bpe_training.txt`（54 行）

階段 3：訓練模型

模型規格

模型結構

=====		
embedding	1,638,400	
block0	778,752	
block1	778,752	
block2	778,752	
block3	778,752	
final_norm	256	
lm_head (與 embedding 共用)	0	

總計	4,753,664	
embedding 佔比	34.5%	

optimizer 拿到 38 個參數張量

`lm_head` 是 0，因為它跟 embedding 共用同一個張量（weight tying）。`optimizer` 拿到 38 個張量——**embedding 就在這 38 個裡面**，跟 `q_proj` 平起平坐。

訓練迴圈

```
# train.py (節錄)
model.zero_grad()
logits = model.forward(x, tr)      # 前向
loss   = ce.forward(logits, y, tr) # 算 loss
dlogits = ce.backward(tr)          # dz = (p - y) / N
model.backward(dlogits, tr)         # 反向，逐層往下

gnorm = opt.clip_grad_norm(args.clip) # 全域梯度裁剪
opt.step(tr, watch='embed.E')        # 這一行才真的改動權重
```

注意 `model.backward()` 和 `opt.step()` 是兩件事。前者只算出「該往哪修」存進梯度表，後者才動權重。

loss 曲線

step	loss	lr	梯度範數	秒
1	8.8149	3.000e-05	0.8127	0.4
200	5.3237	2.992e-03	0.3945	5.7
500	4.2052	2.875e-03	0.3919	13.6
1000	3.6166	2.408e-03	0.3512	26.8
1500	3.1605	1.723e-03	0.3512	40.0
2000	3.0630	1.018e-03	0.3594	53.1
2500	2.9718	4.932e-04	0.3764	66.2
3000	2.7806	3.000e-04	0.3886	79.4

8.81 → 2.78，79 秒。lr 的形狀是 warmup（前 100 步爬升）接 cosine 衰減。

完整曲線每 20 步一筆：`traces/train_log.txt`（152 行）

階段 4：生成

```
python generate.py --prompt "機器學習是"
```

實際輸出

機器學習是一項重要的領域，即將資料能夠從大量資料中學習和推理。
機器學習可以用於學習、自然語言處理和自然語言生成等各種任務。

台灣的天氣如今，在這裡有著令人無法抗衰老的環境和繁華性災難。
這些災難事件可以用於對當地居民群落和探討。

貓是一種常見的寵物，並且常常會受到許多人關注和感悟。
狗通常很快地奔跑、游泳、跳傘……

第一段語意通順、術語正確。後兩段語法沒問題但語意會飄——這就是 475 萬參數、吃 424 萬 token 該有的水準。

（機器學習 被 BPE 合併成單一 token，所以模型對它的處理特別穩。）

階段 5：權重與 tokenizer 綁定

```
python tests/test_tokenizer_binding.py
```

實際輸出

① 正確的 tokenizer loss = 3.5754
② id 對應被打亂 loss = 12.0569

換算成 perplexity（等效於在幾個選項中亂猜）：

正確 tokenizer : 35.7

打亂之後 : 172287.4

完全隨機的模型 : 6400.0

③ 詞表大小不同 -> RuntimeError: size mismatch（安全，馬上知道出事）

④ 擴充 6400 -> 6450，同一句話 loss 幾乎不變

用錯 tokenizer 比完全沒訓練過的模型還糟 27 倍，而且不會報錯。所以 `model.save()` 會把 tokenizer 的指紋一起存進去，載入時比對。

產出的檔案

檔案	行數	大小	看什麼
<code>gradcheck.txt</code>	18	0.9 KB	九層的驗證總表，可直接貼進簡報
<code>model_summary.txt</code>	32	1.1 KB	參數量分佈與所有超參數
<code>bpe_training.txt</code>	54	4.0 KB	詞表怎麼從 5,227 長到 6,400
<code>train_log.txt</code>	152	8.6 KB	每 20 步的 loss / lr / 梯度範數
<code>inference.txt</code>	456	28.6 KB	一次推論的完整前向（沒有反向）
<code>train_step_0001.txt</code>	926	58.8 KB	第 1 步的完整前向 + 反向 + 更新
<code>train_step_3000.txt</code>	925	58.8 KB	最後一步，可跟第 1 步對照
<code>out/model.pt</code>	—	18.6 MB	訓練好的權重
<code>out/tokenizer_bpe.json</code>	—	65 KB	詞表與合併規則

怎麼讀 926 行的追蹤檔

不要從頭讀。先看骨架，再跳到想看的地方。

骨架（行號可直接跳）

```
8  這一批的資料          ← 這批文字長什麼樣
19 前向傳播
25  embed · forward (查表)
45  ===== block0 ===== ← 每個 block 約 100 行
474 softmax + cross-entropy · forward
493 softmax + cross-entropy · backward ← p - y 在這裡
510 反向傳播
776  ===== block0 · backward =====
851  embed · backward (scatter-add) ← 兩條梯度路徑的拆解
889 AdamW 更新（第 1 步）      ← 權重真正被改的地方
917 這一步結束
```

`block1` / `block2` / `block3` 的結構跟 `block0` 一模一樣，看一個就夠。

導覽：五個最值得看的片段

① 第 25 行 — 查表就是查表

公式： $x = E[\text{ids}]$

等價於 $x = \text{one_hot}(\text{ids}) @ E$ ，只是跳過了乘以 0 的部分

`ids` `shape (64, 128)`

`x` `shape (64, 128, 256)`

位置	查哪一列	E[id] 的前 4 維			
t=0	id=4	+0.01647	-0.04764	+0.02504	+0.00472
t=1	id=5868	-0.00623	+0.02665	-0.00391	-0.01012

>> 這一批用到 2036 個不同的 token，詞表共 6400 個。

>> 剩下 4364 列這一步完全沒被碰到——反向時它們的梯度會是 0。

前向沒有任何乘法，就是索引。記住 `2036 / 6400` 這個數字，第 ④ 點會用到。

② 第 493 行 — 整條鏈唯一知道答案的地方

公式: $dz = (p - y) / N$

token id	p	y	$dz = (p-y)/N$	作用
4656	+0.00048	+0.00000	+5.920e-08	壓下去
220	+0.00046	+0.00000	+5.575e-08	壓下去
5868	+0.00026	+1.00000	-0.00012	拉上來（正解）

正解那一格 $y = 1$ ，所以 dz 是**負的**。更新時 $E \leftarrow E - \eta dz$ ，負負得正，分數被推高。其他格全是正的，分數被壓低。

這一行就是整個訓練的全部監督訊號。往下每一層都只是機械式的微分。

③ 第 851 行 — scatter-add，以及它不是唯一的來源

公式: $dE[v] += \sum_{\{t : id_t = v\}} dx[t]$

這一系列的梯度其實是兩條路加起來的：

	來源	$dE[5][0]$	有幾列非零
輸入側（本步 scatter-add）		+0.00125	2036 / 6400 ← 稀疏
輸出側（weight tying）		-0.00420	6400 / 6400 ← 稠密
合計 $dE[5][0]$		-0.00295	6400 / 6400

2036 / 6400 正是第 ① 點那個數字——輸入側只碰得到出現過的 token。但因為 E 同時是分類器權重，softmax 會碰到**每一列**，所以總梯度是稠密的。

注意兩條路的符號還相反（**+0.00125** vs **-0.00420**），合計後變號。

④ 第 889 行 — 位移恰好等於 lr

更新前 p	+0.00388	+0.04323	-0.00344	+0.01698
梯度 g	-1.290e-06	+8.668e-05	-5.620e-05	-6.518e-06
$\hat{m}$	-1.290e-06	+8.668e-05	-5.620e-05	-6.518e-06
$\sqrt{\hat{v}}$	+1.290e-06	+8.668e-05	+5.620e-05	+6.518e-06
$\hat{m}/(\sqrt{\hat{v}}+\epsilon)$	-0.99231	+0.99988	-0.99982	-0.99847
位移 Δ	+2.977e-05	-3.000e-05	+2.999e-05	+2.995e-05
lr				3.000e-05

看第 2 行和第 4 行：梯度從 1.29×10^{-6} 到 8.67×10^{-5} ，**差了 67 倍**。但位移全都是 $\pm 3 \times 10^{-5}$ ，**一模一樣**。

因為第一步時 $\hat{m} = g$ 、 $\sqrt{\hat{v}} = |g|$ ，相除必定是 ± 1 ：

$$\frac{\hat{m}_1}{\sqrt{\hat{v}_1}} = \frac{g}{|g|} = \text{sign}(g)$$

Adam 讓所有參數走一樣遠，差別只在方向。

⑤ 對照 train_step_3000.txt

同一個位置的數字，訓練 3000 步之後：

```
diff <(sed -n '493,500p' traces/train_step_0001.txt) \  
<(sed -n '493,500p' traces/train_step_3000.txt)
```

第 1 步時正解的機率是 0.00026（幾乎是 1/6400 的隨機猜測），第 3000 步會明顯高出來。這是最直接的「模型學到東西了」的證據。

重跑

```
rm -rf out traces  
python tests/test_layers.py  
python train.py --tokenizer bpe --vocab 6400 --steps 3000 \  
--dim 256 --layers 4 --heads 4 --seq 128 --batch 64 \  
--trace-steps 1,3000  
python generate.py --prompt "機器學習是"  
python tests/test_tokenizer_binding.py
```

固定 seed（預設 42），所以數字可以完全重現。

教學文件

程式碼註解裡有完整的公式推導，這裡放的是**程式碼講不了的東西**：流程圖、全景視角、以及做簡報時的敘事順序。

建議順序

	文件	對應程式碼	一句話
0	符號說明	<code>tools/demo_operators.py</code>	先看這份。⊙ 和 @ 都叫「乘」，但完全是兩回事
1	全景圖	—	一句話進去，到權重被改動，中間發生了什麼
2	詞表大小是怎麼決定的	<code>tokenizer/bpe.py</code>	「6400 個詞哪來的」
3	Embedding 與 scatter-add	<code>scratch/embedding.py</code>	查表為什麼是矩陣乘法，反向為什麼是相加
4	反向傳播全鏈	全部	從 <code>p - y</code> 一路走回 <code>E</code>
5	權重是怎麼被改的	<code>scratch/adamw.py</code>	反向算完之後才輪到 optimizer
6	三個最常見的誤解	—	做簡報時最值得講的三件事
7	權重與 tokenizer 綁定	<code>tests/test_tokenizer_binding.py</code>	為什麼 continual pre-training 必須用同一份詞表

單層深入

01-07 是主線敘事。下面三份把單一層拆開講到底：完整的公式推導、結構圖、以及從追蹤檔抓出來的真實數值。做簡報時每一份都可以獨立成一個段落。

	文件	對應程式碼	重點
8	RMSNorm	<code>scratch/rmsnorm.py</code>	完整前向與反向推導（含商法則），以及它為什麼會放大梯度
9	Attention 的結構	<code>scratch/attention.py</code>	六個步驟逐一拆解，含因果遮罩與 softmax 完整 Jacobian
10	FFN 的結構（SwiGLU）	<code>scratch/swiglu.py</code>	閘門機制、為什麼升維再降維、參數量為何佔三分之二

實際跑一遍

	文件	重點
11	完整執行紀錄	從乾淨狀態跑完整條流程。每階段「指令 → 程式碼解說 → 實際輸出」，並附上讀 926 行追蹤檔的導覽

附：怎麼產生講解用的素材

```
# ○ 與 @ 的差別，用最小的矩陣算給你看（含有限差分驗證）
python tools/demo_operators.py

# 逐層梯度驗證表（可直接貼進簡報）
python tests/test_layers.py          # -> traces/gradcheck.txt

# BPE 合併過程，看詞表怎麼從 3653 長到 6400
python tests/test_bpe.py             # -> traces/bpe_training.txt

# 第 1 步的完整數值軌跡（前向 + 反向 + AdamW 每個中間量）
python train.py --trace-steps 1      # -> traces/train_step_0001.txt

# 一次推論的完整前向
python generate.py --prompt "貓是一種" --trace # -> traces/inference.txt

# 證明權重與 tokenizer 綁死（用錯會比沒訓練還糟）
python tests/test_tokenizer_binding.py
```

`traces/sample/` 裡有一份預先跑好的，不用執行就能看。

轉成 PDF

```
python tools/build_pdf.py          # 合併成一份（含封面、目錄）
python tools/build_pdf.py --split  # 每份文件各一個
```

PDF 就輸出在這個目錄，跟 markdown 放在一起。

流程是 markdown → HTML（KaTeX + Mermaid）→ headless Chrome 列印。公式用 KaTeX 渲染、流程圖用 Mermaid 畫，兩者在 PDF 裡都是向量，放大不會糊。

改完文件記得跑 `python tools/check_latex.py` 驗證公式沒被工具鏈破壞。